[University Name]

[Department / School Name]

DBMS PROJECT REPORT

Police Records & Criminal Management System

A centralized database solution for managing police records, criminal cases, arrests, evidence and court proceedings

Submitted by

Arpit Karna (25BCE2887)

G Nikhil Sreenivas (25BCE0178)

Aryan Menon (25BCE0327)

Thota Manoj Sai Kumar (25BAI0021)

Under the guidance of

[Guide Name, Designation]

Academic year: [Academic Year]

Certificate

This is to certify that the project report entitled **"Police Records & Criminal Management System"** is a bonafide record of the work carried out by the following students of the [Department / School Name], [University Name], in partial fulfilment of the requirements of the Database Management Systems course during the academic year [Academic Year]:

- Arpit Karna (25BCE2887)
- G Nikhil Sreenivas (25BCE0178)
- Aryan Menon (25BCE0327)
- Thota Manoj Sai Kumar (25BAI0021)

The work embodied in this report was carried out under the guidance of [Guide Name, Designation] and has not been submitted elsewhere for any other course or degree.

[Guide Name] - Project Guide	[Name] - Head of the Department	[Name] - Examiner

Date: [DD/MM/YYYY] Place: [Place]

Declaration

We hereby declare that the project report entitled **"Police Records & Criminal Management System"** is our own work, carried out under the guidance of [Guide Name, Designation]. The design, source code, database, tests and screenshots described in this report were produced by the project team. All third-party software, libraries and documentation that we used are acknowledged in the References section, and no part of this report has been copied from another source or submitted for any other course.

- Arpit Karna (25BCE2887)
- G Nikhil Sreenivas (25BCE0178)
- Aryan Menon (25BCE0327)
- Thota Manoj Sai Kumar (25BAI0021)

[Signature]	[Signature]	[Signature]	[Signature]

Date: [DD/MM/YYYY] Place: [Place]

Acknowledgement

We express our sincere gratitude to our project guide, [Guide Name, Designation], for the guidance, encouragement and constructive feedback that shaped this project from the first review to the final report.

We thank the Head of the Department, [HOD Name], and the faculty of the [Department / School Name], [University Name], for providing the knowledge and facilities that made this work possible.

We also thank the maintainers of the open-source tools on which PCRMS is built (MySQL, Node.js, Express, React, Vite and the other libraries listed in the References) and our families and friends for their support.

Project team

- Arpit Karna (25BCE2887)
- G Nikhil Sreenivas (25BCE0178)
- Aryan Menon (25BCE0327)
- Thota Manoj Sai Kumar (25BAI0021)

Abstract

Manual, paper-based record keeping in police stations leads to duplicated records, slow searching, weak case tracking and time-consuming reporting. The Police Records & Criminal Management System (PCRMS) addresses these problems with a centralized relational database and a role-based web application. The database is a thirteen-table MySQL 8 schema in third normal form, with primary, foreign and unique keys, enumerated types and indexes, three views, three stored procedures and four triggers. The triggers and procedures implement the case lifecycle inside the database: recording an arrest marks the criminal as arrested and moves the case to Under Investigation, scheduling a hearing moves it to In Court, evidence receives a chain-of-custody log, and every status change is audited. A Node.js/Express REST API authenticates users with bcrypt and JSON Web Tokens and enforces three roles (viewer, officer, administrator) using parameterized SQL. A responsive React interface provides a dashboard, nine record-management modules, case and criminal detail pages, global search and six exportable reports. The system was verified with 44 automated API and database tests and 24 user-interface tests, all of which passed, and the screenshots in this report were captured from the running application. Known limitations include HTTP 500 responses for invalid input, no user-management screens and no pagination. PCRMS demonstrates how DBMS concepts can be applied to a realistic records-management problem.

Keywords: DBMS, MySQL, normalization, stored procedures, triggers, REST API, React, police records management.

Table of Contents

Chapter 1: Introduction 1
1.1 Project Background 1
1.2 Problem Statement 1
1.3 Motivation 1
1.4 Aim and Objectives 1
1.5 Scope of the Project 2
1.6 Proposed Solution 2
1.7 Key Features and Expected Benefits 2
1.8 Organization of the Report 3
Chapter 2: Existing System and Proposed System 4
2.1 Existing Approach and Its Limitations 4
2.2 Proposed System 4
2.3 Comparison between Existing and Proposed Systems 4
2.4 Functional Requirements 5
2.5 Non-Functional Requirements 6
2.6 Hardware and Software Requirements 6
2.7 Assumptions and Constraints 7
Chapter 3: System Analysis and Design 8
3.1 Overall System Architecture 8
3.2 Component and Module Structure 8
3.3 Data Flow Diagrams 9
3.4 Use Case Diagram 11
3.5 Activity Diagram 12
3.6 Sequence Diagram 13
3.7 Class Diagram 14
3.8 Choice of Technologies 15
Chapter 4: Database Design 16
4.1 Database Technology and Design Approach 16
4.2 List of Tables 16
4.3 Entity-Relationship Diagram 17
4.4 Relationships and Cardinalities 19
4.5 Table Schemas 19
4.5.1 PoliceStation 20
4.5.2 Officer 20
4.5.3 Criminal 20
4.5.4 Cases 21
4.5.5 Complaint 21
4.5.6 CriminalCase 21
4.5.7 Arrest 22
4.5.8 Evidence 22
4.5.9 EvidenceCustody 23
4.5.10 Court 23
4.5.11 CourtProceeding 23

4.5.12 AppUser 23
4.5.13 CaseStatusLog 24
4.6 Constraints and Indexes 24
4.7 Normalization Analysis 25
4.8 Views, Stored Procedures and Triggers 26
4.9 How Data Is Stored, Retrieved, Updated and Deleted 27

Chapter 5: System Implementation 28
5.1 Project Structure 28
5.2 Frontend Architecture 29
5.3 Backend Architecture 30
5.4 API Endpoints 31
5.5 Database Integration 31
5.6 Authentication and Authorization 31
5.7 Input Validation and Error Handling 32
5.8 Core Business Logic 33
5.9 Third-Party Libraries and Dependencies 34
5.10 Integration of Frontend, Backend and Database 34
5.11 Important Implementation Decisions 34

Chapter 6: User Interface and Project Screenshots 36
6.1 Login Screen 36
6.2 Dashboard 37
6.3 Lists, Search and Filtering 38
6.4 Detail Pages 40
6.5 Forms and Data Entry 42
6.6 Reports 44
6.7 Empty and Error States 45
6.8 Role-Based Interface 46
6.9 Responsive and Mobile Layout 47
6.10 Screens That Are Not Available 50

Chapter 7: Testing and Validation 51
7.1 Testing Strategy 51
7.2 Test Environment 51
7.3 API and Database Test Cases 51
7.4 User-Interface Test Cases 53
7.5 Summary of Results 55
7.6 Observations and Defects 55
7.7 Tests Not Performed 55

Chapter 8: Results and Discussion 57
8.1 Summary of Implemented Features 57
8.2 Results Observed during Testing 57
8.3 How the Application Addresses the Original Problem 57
8.4 Limitations and Unresolved Issues 58

Chapter 9: Conclusion and Future Scope 59
9.1 Summary of the Work 59
9.2 Practical Value 59

9.3 Limitations 59
9.4 Future Scope 59
References **61**
Appendix A: API Endpoint Summary **62**
Appendix B: Database Schema Reference **64**
Appendix C: Additional Screenshots **65**
Appendix D: Source Code and Configuration Notes **70**
Appendix E: Setup and Execution Instructions **71**
Appendix F: Glossary **72**

List of Figures

Figure 1: Three-tier system architecture of PCRMS 8
Figure 2: Component diagram: frontend, API and database modules 9
Figure 3: Level 0 (context) data flow diagram 10
Figure 4: Level 1 data flow diagram 11
Figure 5: Use case diagram with actors and role inheritance 12
Figure 6: Activity diagram of the case lifecycle workflow 13
Figure 7: Sequence diagram: recording an arrest 14
Figure 8: Class/module diagram of the application code 15
Figure 9: Entity-relationship diagram of the PCRMS database (Chen notation) 18
Figure 10: Login screen 36
Figure 11: Login error state after an incorrect password 37
Figure 12: Dashboard with statistics, charts and upcoming hearings 38
Figure 13: Cases / FIR list 39
Figure 14: Cases filtered by status "In Court" 39
Figure 15: Global search results 40
Figure 16: Case detail page 41
Figure 17: Criminal profile page 41
Figure 18: Criminals list 42
Figure 19: Add-arrest form with foreign-key dropdowns 43
Figure 20: Chain-of-custody dialog 44
Figure 21: Reports page: case overview 45
Figure 22: Reports page: officer workload 45
Figure 23: Empty state when no record matches 46
Figure 24: Database constraint error shown in the interface 46
Figure 25: Cases page as seen by the read-only viewer role 47
Figure 26: Dashboard on a 390 px wide screen 48
Figure 27: Cases list on a 390 px wide screen 49
Figure 28: Navigation drawer on a 390 px wide screen 50
Figure 29: Complaints list 66
Figure 30: Arrests list 66
Figure 31: Evidence list 67
Figure 32: Court proceedings list 67
Figure 33: Officers list 68
Figure 34: Police stations list 68
Figure 35: Courts list 69

List of Tables

Table 1: Comparison of manual record keeping with PCRMS 4
Table 2: Functional requirements and their status 5
Table 3: Non-functional requirements and how they are addressed 6
Table 4: Development and test environment (as used) 6
Table 5: Software requirements to run PCRMS 6
Table 6: Technology choices and rationale 15
Table 7: The thirteen tables of the pcrms database 16
Table 8: Relationships between entities 19
Table 9: Schema of table PoliceStation 20
Table 10: Schema of table Officer 20
Table 11: Schema of table Criminal 20
Table 12: Schema of table Cases 21
Table 13: Schema of table Complaint 21
Table 14: Schema of table CriminalCase 22
Table 15: Schema of table Arrest 22
Table 16: Schema of table Evidence 22
Table 17: Schema of table EvidenceCustody 23
Table 18: Schema of table Court 23
Table 19: Schema of table CourtProceeding 23
Table 20: Schema of table AppUser 24
Table 21: Schema of table CaseStatusLog 24
Table 22: Indexes in the pcrms database 24
Table 23: Programmability objects in the database 26
Table 24: Main source files and their sizes 28
Table 25: Special (non-CRUD) API routes 31
Table 26: Permissions by role 32
Table 27: Mapping of failures to HTTP responses 32
Table 28: Runtime and build dependencies (versions from package-lock.json) 34
Table 29: Key design decisions and trade-offs 34
Table 30: API and database test cases and results (all executed) 51
Table 31: User-interface test cases and results (all executed) 54
Table 32: Summary of executed tests 55
Table 33: Proposed tests that were not executed 55
Table 34: Objectives and the evidence that they were met 57
Table 35: Problems from the project review and the corresponding solution 57
Table 36: Special API routes 62
Table 37: Generated CRUD routes (9 resources x 5 routes) 62
Table 38: Foreign-key constraints 64
Table 39: Storage engine and collation 64
Table 40: Glossary 72

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DBMS	Database Management System
DDL	Data Definition Language
DFD	Data Flow Diagram
ER	Entity-Relationship
FIR	First Information Report
FK	Foreign Key
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
M:N	Many-to-Many
PK	Primary Key
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single-Page Application
SQL	Structured Query Language
UI	User Interface
UK	Unique Key
1NF / 2NF / 3NF	First / Second / Third Normal Form

Chapter 1: Introduction

1.1 Project Background

Police departments create and consult a very large amount of structured information: stations and the officers posted to them, citizen complaints, First Information Reports (FIRs), the cases that follow, the people involved, arrests, seized evidence and the court hearings that decide each case. These facts are tightly related. An arrest belongs to a case; evidence belongs to an arrest; a hearing belongs to a case and a court. Keeping them consistent is the kind of problem that relational database management systems (DBMS) were designed to solve [14].

The Police Records & Criminal Management System (PCRMS) is a database-centred web application built as a DBMS course project. It stores this information in a normalized MySQL database and exposes it through a role-based web interface so that authorized users can record, search and cross-reference police data from one place.

1.2 Problem Statement

The team's project review document identifies four problems with traditional, manual and paper-based record keeping in police stations: **data duplication** (the same person or case recorded separately and inconsistently), **difficult searching** (no unified way to find a criminal or case file), **poor case tracking** (no structured status updates, so cases can be delayed or lost) and **time-consuming management** (manual updates and report generation consume officer time). PCRMS is designed to address exactly these four problems.

1.3 Motivation

A police records system is a good vehicle for applying core DBMS concepts to a realistic scenario: many interrelated entities, a many-to-many relationship (criminals and cases), a strict lifecycle (complaint to FIR to arrest to evidence to hearing to closure), integrity rules that must never be broken, and a need for audit trails. The scenario also requires application-level concerns such as authentication and role-based authorization, which show how a database is used inside a complete system rather than in isolation.

1.4 Aim and Objectives

Aim: to design and implement a centralized, normalized database and a web application that manages police records, criminals, cases, arrests, evidence and court proceedings.

The objectives, as stated in the project review document and refined during implementation, are:

1. Centralize police records in one secure relational database with enforced primary and foreign keys.
2. Manage criminals, cases, arrests and evidence digitally, including a chain-of-custody log for evidence.
3. Track court proceedings and generate reports instantly from live data.
4. Provide fast searching and cross-referencing across related entities.
5. Automate lifecycle rules inside the database with stored procedures and triggers so that statuses stay consistent.
6. Restrict what each user can do through authenticated, role-based access (administrator, officer, viewer).

1.5 Scope of the Project

In scope: nine modules (police stations, officers, criminals, cases/FIR, complaints, arrests, evidence, court proceedings, reports), the thirteen-table database, a REST API, a React single-page application, authentication with three roles, dashboard statistics, six reports with CSV export, global search, and automated API/database and UI tests.

Out of scope (not implemented): user registration and account-management screens (the three demo accounts are created by the setup script), file upload for photographs (a photo URL field is stored), a mobile application, cloud deployment, biometric integration, notifications and analytics beyond the implemented reports. These are discussed as future work in Chapter 9.

1.6 Proposed Solution

PCRMS is a three-tier application (Figure 1). A MySQL 8 database holds the data and enforces integrity; business rules that must hold regardless of the client are implemented inside the database as stored procedures and triggers. A Node.js/Express API authenticates users, checks their role and runs parameterized SQL. A React single-page application provides the user interface. The same schema, API and interface cover the complete case lifecycle shown in Figure 6.

1.7 Key Features and Expected Benefits

- **Single source of truth:** one database replaces separate paper registers, removing duplicated records.
- **Integrity by design:** foreign keys, unique keys and enumerated types prevent orphaned or invalid records; the test suite confirms that referenced rows cannot be deleted and duplicate FIR numbers are rejected.

- **Automatic status tracking:** recording an arrest marks the criminal as Arrested and moves the case to Under Investigation; scheduling a hearing moves it to In Court; every change is logged.
- **Evidence accountability:** each evidence item has a timestamped chain-of-custody log.
- **Fast retrieval:** per-module search, a global search, a status filter and criminal and case detail pages with related records.
- **Instant reporting:** dashboard totals and six reports that can be exported as CSV or printed.
- **Role-based access:** viewers can only read, officers cannot delete, administrators have full access.

1.8 Organization of the Report

Chapter 2 compares the existing manual approach with the proposed system and lists requirements. Chapter 3 presents the architecture and UML/DFD diagrams. Chapter 4 documents the database design, including the ER diagram, table schemas and normalization. Chapter 5 describes the implementation. Chapter 6 shows the running application. Chapter 7 reports the tests that were executed and their results. Chapter 8 discusses the results and limitations, and Chapter 9 concludes and outlines future work. The appendices contain the API reference, schema reference, additional screenshots, configuration notes, setup instructions and a glossary.

Chapter 2: Existing System and Proposed System

2.1 Existing Approach and Its Limitations

The starting point for this project is manual, paper-based record keeping, as described in the team's project review document. No field study or statistics were collected for this report, so the limitations below are those identified in that document and the logical consequences of keeping related records on separate paper registers or unconnected files:

- **Redundant records:** the same person or case can be written down in more than one register or station, with no mechanism to keep the copies consistent.
- **Slow, error-prone search:** finding a criminal or case means looking through registers or files by hand; there is no cross-referencing from a person to their cases, arrests and evidence.
- **Weak case tracking:** without a structured status field and a history of changes, it is hard to know where a case stands or when its state changed.
- **Manual reporting:** statistics and reports must be compiled by hand.
- **No enforced integrity or access control:** nothing prevents an arrest from referring to a case that does not exist, or limits who may change or remove a record.

2.2 Proposed System

PCRMS replaces separate registers with one relational database and an application on top of it. The database stores the entities and relationships of Figure 9; the API enforces authentication and roles; the interface offers lists, forms, detail pages, search, a dashboard and reports. Lifecycle rules (status changes, custody log entries, audit rows) are executed by the database itself, so every client sees the same behaviour.

2.3 Comparison between Existing and Proposed Systems

Table 1: Comparison of manual record keeping with PCRMS

Aspect	Manual / paper-based approach	PCRMS (as implemented)
Data storage	Separate registers or files	One MySQL database, 13 related tables
Duplicate records	Possible; copies can disagree	Single copy referenced by key; UNIQUE constraints on FIR number, badge number, username
Searching	Manual look-up	Per-module search, global search, status filter, indexes on common search columns
Cross-referencing	Manual	Joins through foreign keys; criminal and case detail pages show related records

Aspect	Manual / paper-based approach	PCRMS (as implemented)
Case status tracking	Informal	Status field with fixed values, changed by triggers and procedures, every change logged
Evidence handling	Paper log	EvidenceCustody table with officer and timestamp for each action
Reports	Compiled by hand	Six live reports, dashboard totals, CSV export and print
Integrity	Depends on staff discipline	Primary, foreign, unique and NOT NULL constraints enforced by the DBMS
Access control	Physical access	Login with bcrypt-hashed passwords, JWT, three roles
Availability of information	One physical copy	Any authorized browser session that can reach the server

2.4 Functional Requirements

The functional requirements below were derived from the project review document. The right-hand column states whether each requirement is implemented and where it is verified in Chapter 7.

Table 2: Functional requirements and their status

ID	Requirement	Status / verification
FR-01	Users log in with a username and password and receive a session token.	Implemented; TC-01, TC-02, UI-03
FR-02	Access is restricted by role: viewer (read only), officer (no delete), administrator (all).	Implemented; TC-08, TC-10, TC-11
FR-03	Create, view, edit and delete police stations, officers and courts.	Implemented; TC-12 to TC-17
FR-04	Maintain criminal profiles (name, date of birth, gender, address, status, photo link).	Implemented; UI-07
FR-05	Register an FIR so that a case is created and an optional complaint is linked to it.	Implemented; TC-24
FR-06	Manage cases, filter them by status and close a case.	Implemented; TC-16, TC-32, UI-18
FR-07	Link and unlink criminals to a case (many-to-many) with a role.	Implemented; TC-34
FR-08	Record citizen complaints at a station.	Implemented; UI-06
FR-09	Record arrests; update criminal and case status automatically.	Implemented; TC-26
FR-10	Record evidence and keep a chain-of-custody log.	Implemented; TC-29, TC-30
FR-11	Record court proceedings; moving a case to In Court automatically.	Implemented; TC-31
FR-12	Keep a history of case status changes.	Implemented; TC-28
FR-13	Search records per module and across criminals, cases and officers.	Implemented; TC-44, UI-21
FR-14	Show a dashboard with totals, charts and upcoming hearings.	Implemented; TC-36, UI-04
FR-15	Provide reports with CSV export and print.	Implemented; TC-37 onwards, UI-20
FR-16	Show a criminal's case history and arrests.	Implemented; TC-33, UI-15

ID	Requirement	Status / verification
FR-17	Usable on desktop and mobile screen sizes.	Implemented; UI-24

2.5 Non-Functional Requirements

Table 3: Non-functional requirements and how they are addressed

Quality	How it is addressed in the implementation	Evidence
Security	bcrypt password hashes; JWT with 8-hour expiry; role check on every API request; parameterized SQL; only whitelisted columns are written.	TC-06, TC-23
Data integrity	Primary, foreign and unique keys, NOT NULL, ENUM types; transactional FIR registration with rollback.	TC-22, TC-25
Usability	Consistent layout, search on every list, labelled forms, loading, empty and error states, demo-account shortcuts on the login page.	Chapter 6
Maintainability	Nine modules share one generic CRUD router and one generic list/form page, each driven by a small configuration object.	Section 5.3
Portability	Node.js and MySQL run on Windows, Linux and macOS. Only Windows 11 was used for development and testing.	Section 7.2
Responsiveness	Responsive CSS with a slide-in navigation drawer below 900 px.	UI-24
Performance	Connection pool (10), indexes on frequently filtered columns. No load or performance testing was carried out, so no performance figures are claimed.	Section 7.7
Accessibility	Semantic landmarks, labelled controls and keyboard-closable drawer. No formal accessibility audit was performed.	Section 7.7

2.6 Hardware and Software Requirements

Table 4: Development and test environment (as used)

Item	Specification
Operating system	Windows 11 Home (10.0.26200)
Runtime	Node.js v22.19.0, npm 10.9.3
Database server	MySQL Community Server 8.4.9
Browser used for testing	Google Chrome 154.0.8037.98 (headless, via Puppeteer Core)
Editor / tooling	Visual Studio Code, Git

Table 5: Software requirements to run PCRMS

Component	Requirement
Server runtime	Node.js 18 or later (tested with 22.19.0)
Database	MySQL 8.x (tested with 8.4.9); InnoDB; utf8mb4
Client browser	A current evergreen browser with JavaScript enabled (tested with Chrome)
Ports	3306 (MySQL), 4000 (API), 5173 (development UI)

No minimum hardware figures were measured. Any machine able to run Node.js and MySQL 8 and a modern browser is sufficient for the demonstration data set.

2.7 Assumptions and Constraints

- The system runs on a trusted network; HTTPS termination is not configured by the project.
- User accounts are created by the setup script. There is no self-registration, to avoid unauthorized sign-ups to a records system.
- Sample data is fictional and non-sensitive.
- All authenticated users can read all records; there is no per-station data partitioning.
- Photographs are stored as URLs, not uploaded files.
- The system was built within the timeline of a single course project by a four-member team.

Chapter 3: System Analysis and Design

3.1 Overall System Architecture

PCRMS follows a classic three-tier architecture, shown in Figure 1. The presentation tier is a React 18 single-page application built with Vite [1][2]. It communicates with the application tier, a Node.js/Express REST API [6], using JSON over HTTP and a JWT bearer token. The API talks to MySQL 8 [7] through a mysql2 connection pool [8] using parameterized queries. During development the Vite server proxies requests under /api to the API on port 4000, so the browser sees a single origin.

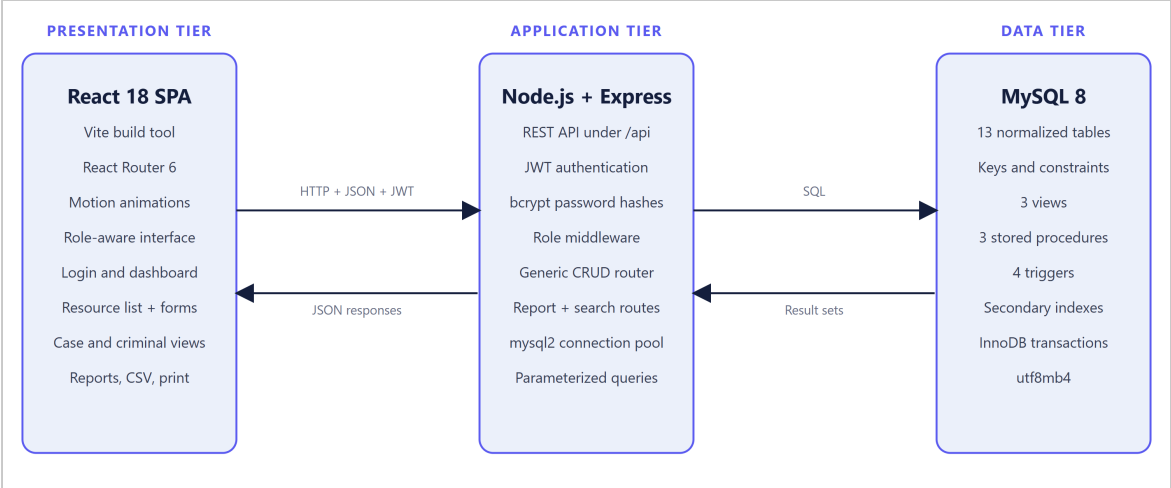


Figure 1: Three-tier system architecture of PCRMS

The data tier is not a passive store. Views, stored procedures and triggers implement report queries, FIR registration and status automation inside the database, so the rules hold for every client.

3.2 Component and Module Structure

Figure 2 shows how the modules interact. Each page of the frontend calls one family of API endpoints: the generic resource pages use the generic CRUD router, the case and criminal detail pages use the special endpoints, and the dashboard and reports use the report endpoints. All routes pass through the authentication and role middleware before reaching the database.

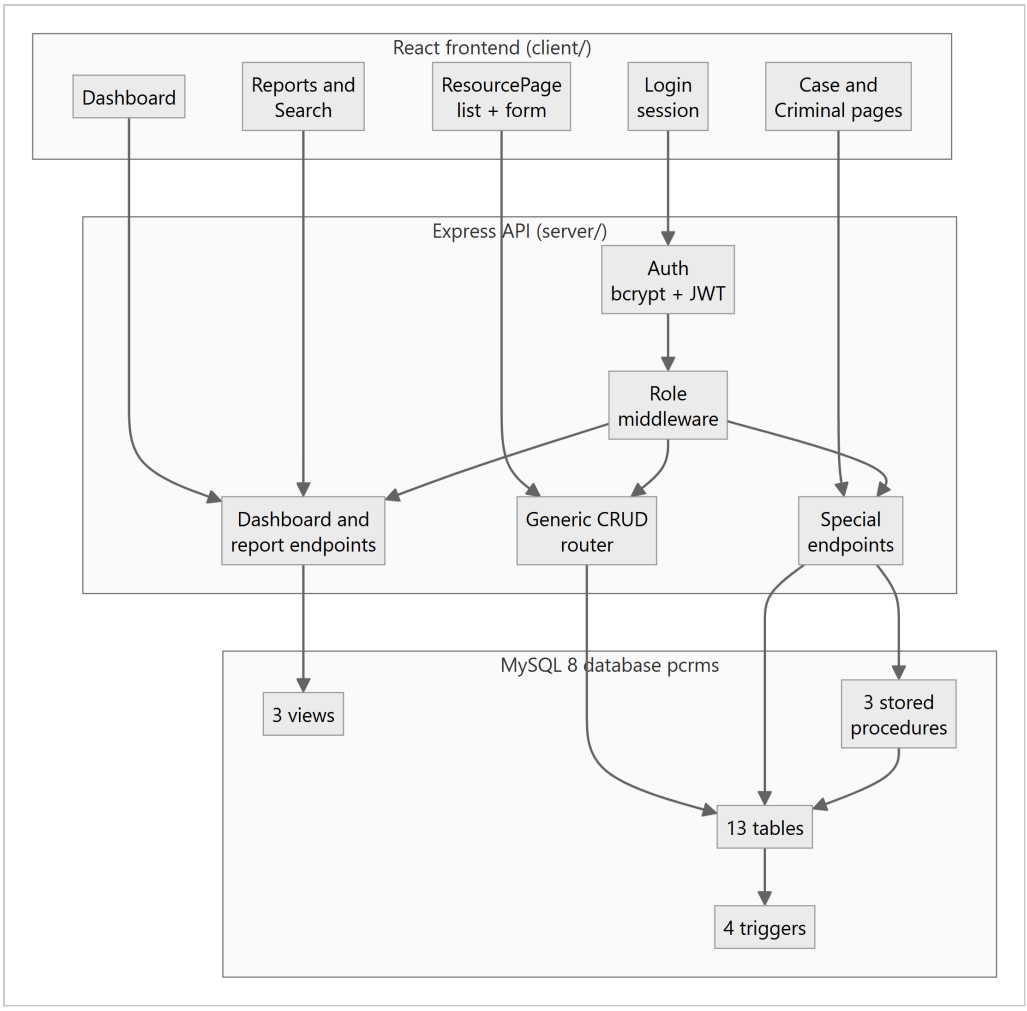


Figure 2: Component diagram: frontend, API and database modules

3.3 Data Flow Diagrams

Figure 3 is the context-level DFD. PCRMS is a single process exchanging data with three external entities: the administrator (master data and deletions), the police officer (complaints, FIRs, arrests, evidence and hearings) and the viewer (searches and report requests).

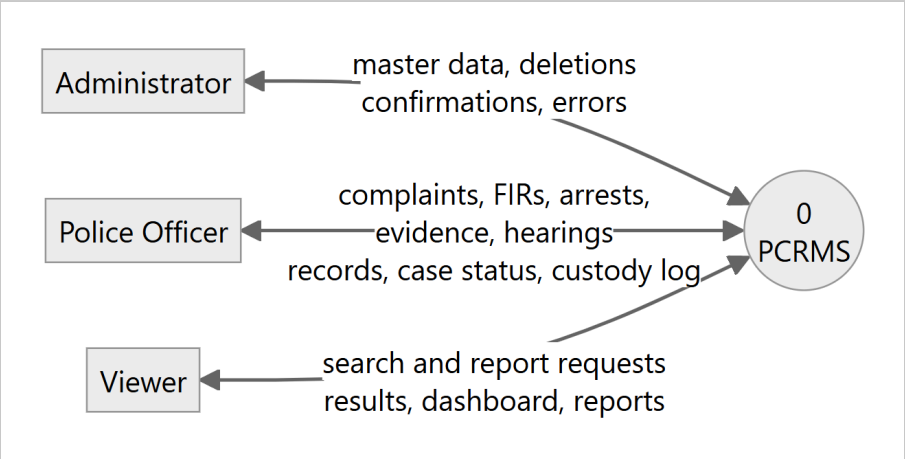


Figure 3: Level 0 (context) data flow diagram

Figure 4 decomposes the system into six processes: authentication and authorization (1.0), master data (2.0), complaint and FIR registration (3.0), arrest and evidence (4.0), court proceedings (5.0) and search, dashboard and reports (6.0). Data stores D1 to D5 group the thirteen tables by subject. Dotted arrows show database triggers that update case status as a side effect of arrests and hearings. Process 1.0 guards every other process; for readability its authorization flows to the other processes are not drawn individually.

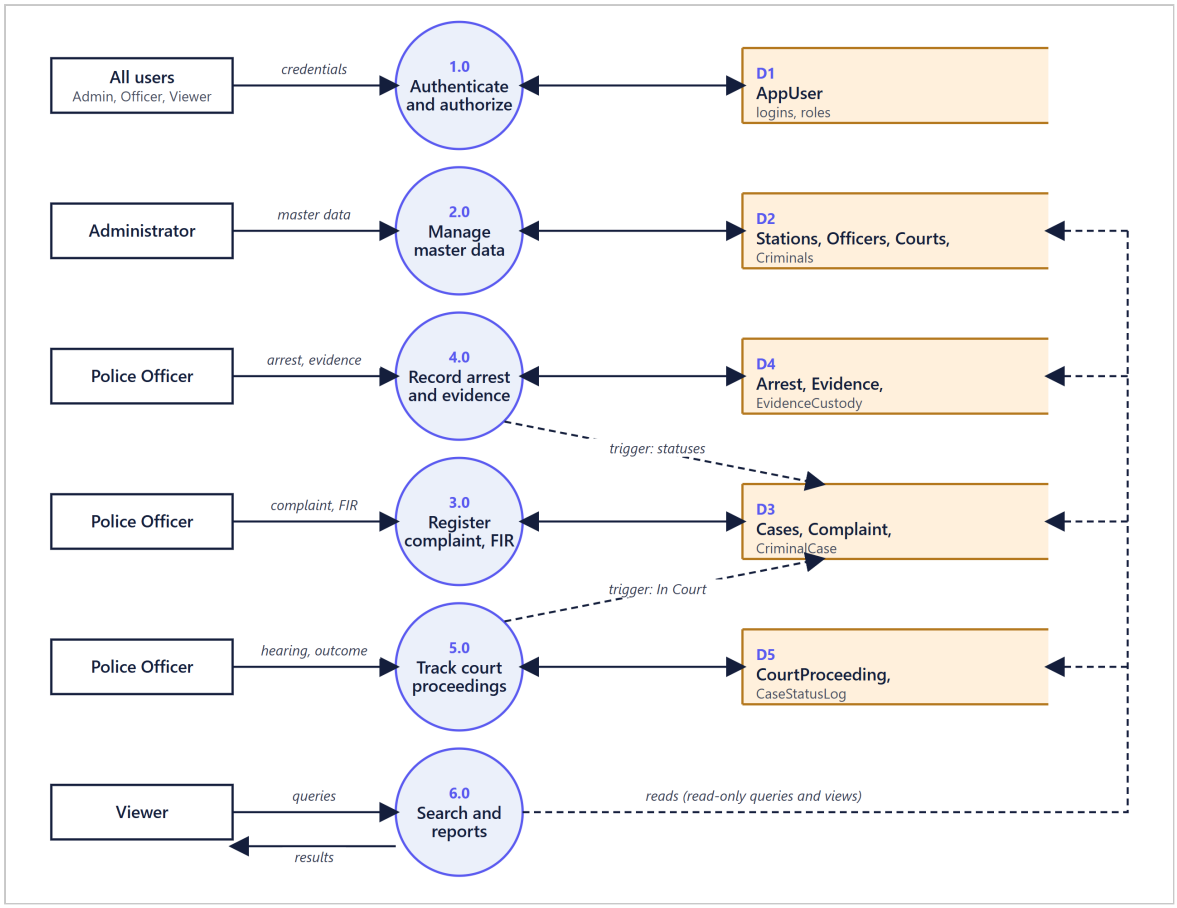


Figure 4: Level 1 data flow diagram

3.4 Use Case Diagram

Figure 5 shows the actors and use cases. The roles form a hierarchy that mirrors the implemented permissions: the **viewer** can log in, view the dashboard, search, read reports and view details; the **police officer** inherits these and can create and edit records (FIRs, complaints, criminals, arrests, evidence, custody, hearings and master data); the **administrator** inherits everything and can additionally delete records.

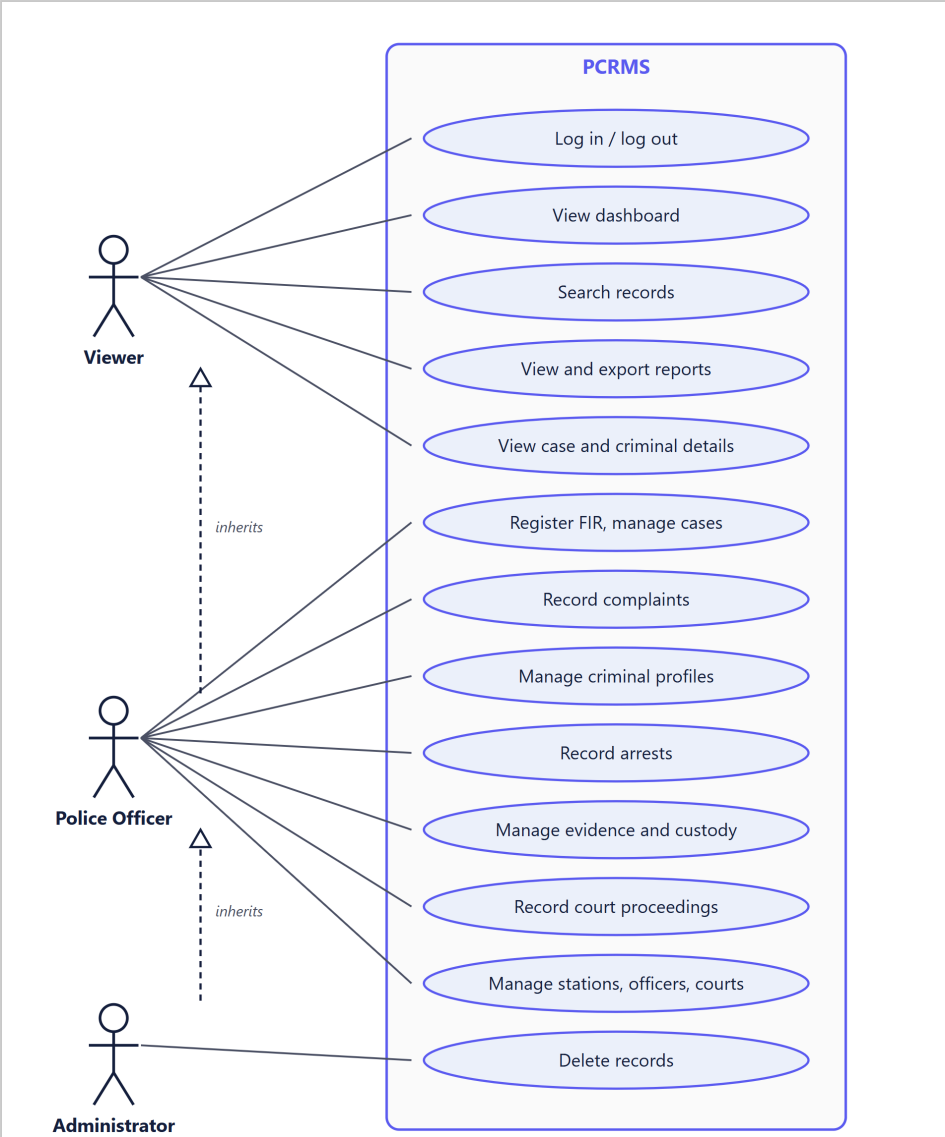


Figure 5: Use case diagram with actors and role inheritance

3.5 Activity Diagram

Figure 6 follows the main workflow from login to case closure. After a successful login the officer records a complaint and registers an FIR, which creates the case. An arrest triggers automatic status changes; evidence is added with its first custody entry; when a hearing is recorded the case moves to In Court; closing the case completes the lifecycle. The loops show that investigation and custody logging can continue until an arrest or hearing occurs.

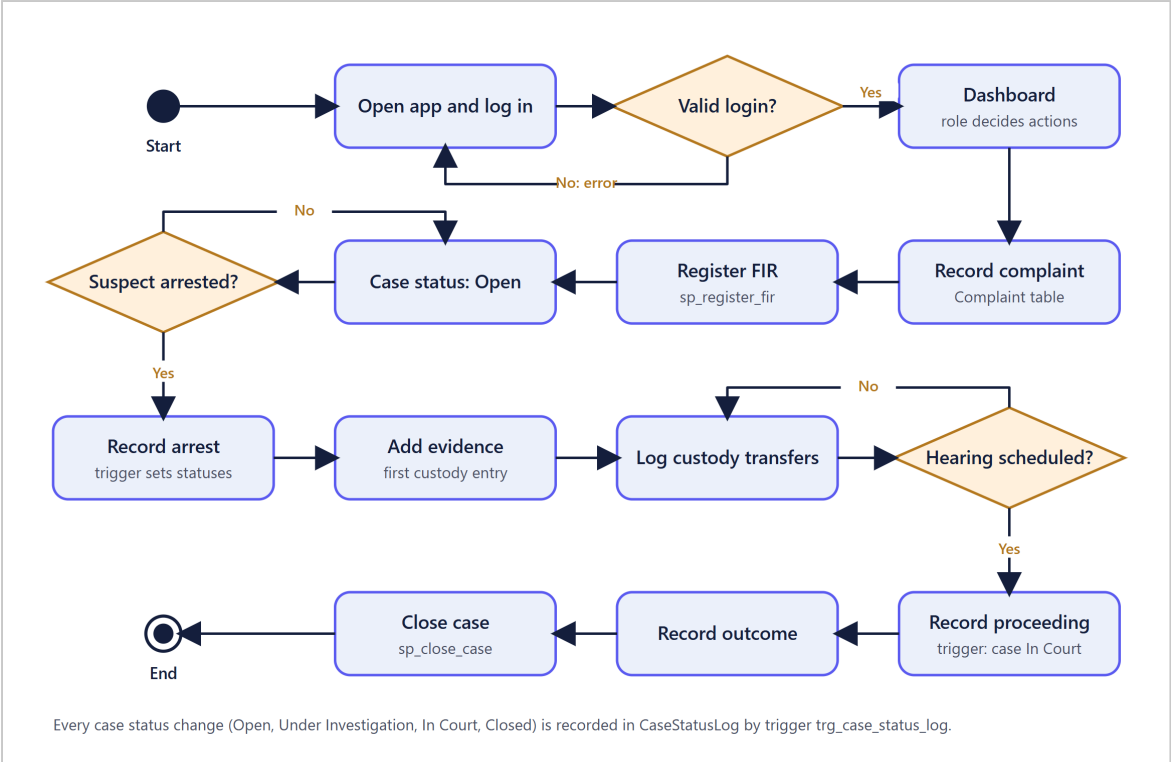


Figure 6: Activity diagram of the case lifecycle workflow

3.6 Sequence Diagram

Figure 7 shows the end-to-end operation "record an arrest". The request is rejected with 401 if the token is invalid and with 403 if the role is not permitted. Otherwise a parameterized INSERT is executed; the trigger trg_arrest_after_insert runs inside the same statement and updates the criminal and the case, and the trigger on Cases writes a status-log row. The interface then reloads the list.

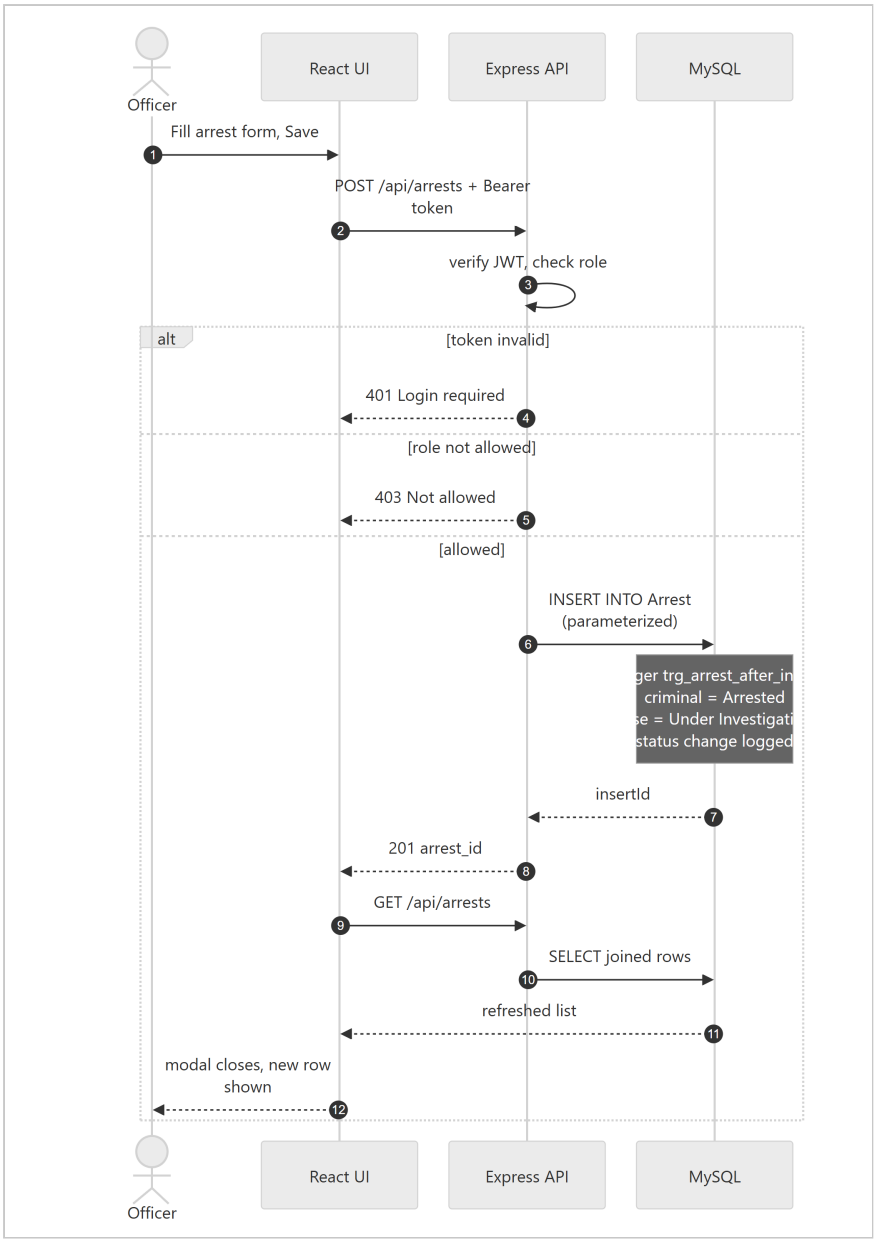


Figure 7: Sequence diagram: recording an arrest

3.7 Class Diagram

PCRMS is not an object-relational application: its persistent model is the relational schema in Chapter 4, and the API is written in a functional, configuration-driven style. A class diagram of domain objects would therefore duplicate the ER diagram. Figure 8 instead shows the main code-level classes and modules: the React components, the API client, the Express server, the resource configuration and the connection pool, and how they depend on each other.

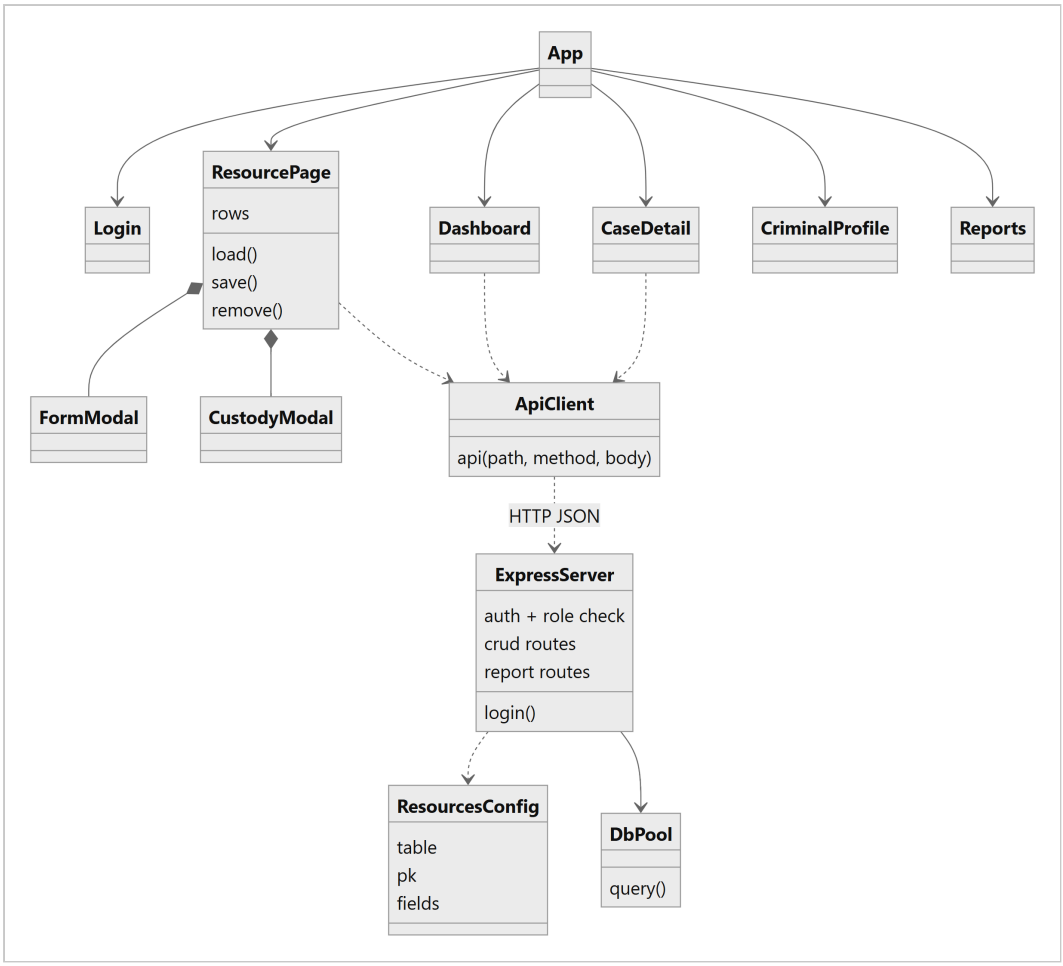


Figure 8: Class/module diagram of the application code

3.8 Choice of Technologies

Table 6: Technology choices and rationale

Technology	Role	Reason
MySQL 8.4	Relational database	Named in the project plan; supports InnoDB transactions, foreign keys, stored procedures, triggers and views.
Node.js + Express 4	REST API	Same language as the frontend; small, well-documented framework; easy to generate CRUD routes from configuration.
mysql2	Database driver	Promise API, prepared/parameterized queries, connection pool.
React 18 + Vite	Frontend	Component model suits repeated list/form pages; fast development server and build.
React Router 6	Client-side routing	Nested routes and navigation without page reloads.
Motion for React	Animations	Page transitions, staggered lists and hover effects using one small library.
JWT + bcryptjs	Authentication	Stateless tokens, adaptive password hashing in pure JavaScript.

Chapter 4: Database Design

4.1 Database Technology and Design Approach

The database is a relational schema named pcrms on MySQL 8.4.9. All thirteen tables use the InnoDB engine and the utf8mb4 character set, which provides transactions and foreign-key enforcement. The design started from the entities named in the project review (police station, officer, criminal, case, arrest, evidence, court, court proceeding) and followed the entity-relationship approach [15]: identify entities and their attributes, identify relationships and their cardinalities, resolve many-to-many relationships with a junction table, and normalize to third normal form [14]. The final schema was then created with the DDL in database/schema.sql and the facts in this chapter were read back from the running database using information_schema, so they describe the implemented schema rather than the plan.

Four tables were added to the nine in the review document, because the modules in the review needed them: **Complaint** (the Complaints module), **EvidenceCustody** (the "chain of custody" in the Evidence module), **AppUser** (role-based login) and **CaseStatusLog** (status history populated by a trigger). Two foreign keys were also added to make relationships real: Arrest.criminal_id (who was arrested) and CourtProceeding.court_id (where the hearing took place).

4.2 List of Tables

Table 7: The thirteen tables of the pcrms database

#	Table	Purpose
1	PoliceStation	Police stations with address, city and phone. Parent of officers and complaints.
2	Officer	Police officers with unique badge number, rank and the station they belong to.
3	Criminal	Persons of interest with personal details, photo link and a status (Wanted, Arrested, Convicted, Released).
4	Cases	Cases created from an FIR. Holds FIR number, type, status and the officer in charge. Named Cases because CASE is a reserved SQL word.
5	Complaint	Citizen complaints received at a station; optionally linked to the case they led to.
6	CriminalCase	Junction table that resolves the many-to-many relationship between criminals and cases; stores the person's role in the case.
7	Arrest	An arrest of a criminal in connection with a case by an officer, with date, location and charges.
8	Evidence	Evidence items collected for an arrest.
9	EvidenceCustody	Chain-of-custody log: each hand-over or action on an evidence item, with the officer and timestamp.
10	Court	Courts with name and location.
11	CourtProceeding	Hearings of a case in a court, with date and outcome (NULL until concluded).

#	Table	Purpose
12	AppUser	Application logins with bcrypt password hash and role (admin, officer, viewer); optionally linked to an officer.
13	CaseStatusLog	Audit rows written by a trigger whenever a case's status changes.

In addition to the tables, the database contains 3 views (v_case_overview, v_station_stats, v_wanted_criminals), 3 stored procedures (sp_close_case, sp_criminal_history, sp_register_fir) and 4 triggers (trg_arrest_after_insert, trg_proceeding_after_insert, trg_case_status_log, trg_evidence_custody_init).

4.3 Entity-Relationship Diagram

Figure 9 is the entity-relationship diagram of the implemented database, drawn in Chen notation: rectangles are entities, diamonds are relationships, ellipses are attributes and an underlined attribute is the primary key (a dashed underline marks a unique key). A single line means partial participation and a double line means total participation (every row of that entity must take part, which corresponds to a NOT NULL foreign key). An arrow points to the "one" side of a many-to-one relationship; a relationship drawn without arrows is many-to-many. Foreign-key columns are shown as relationships and not repeated as attributes. The diagram has 12 entities, 15 relationships and every other column as an attribute, and was checked against the live database schema: no column is missing.

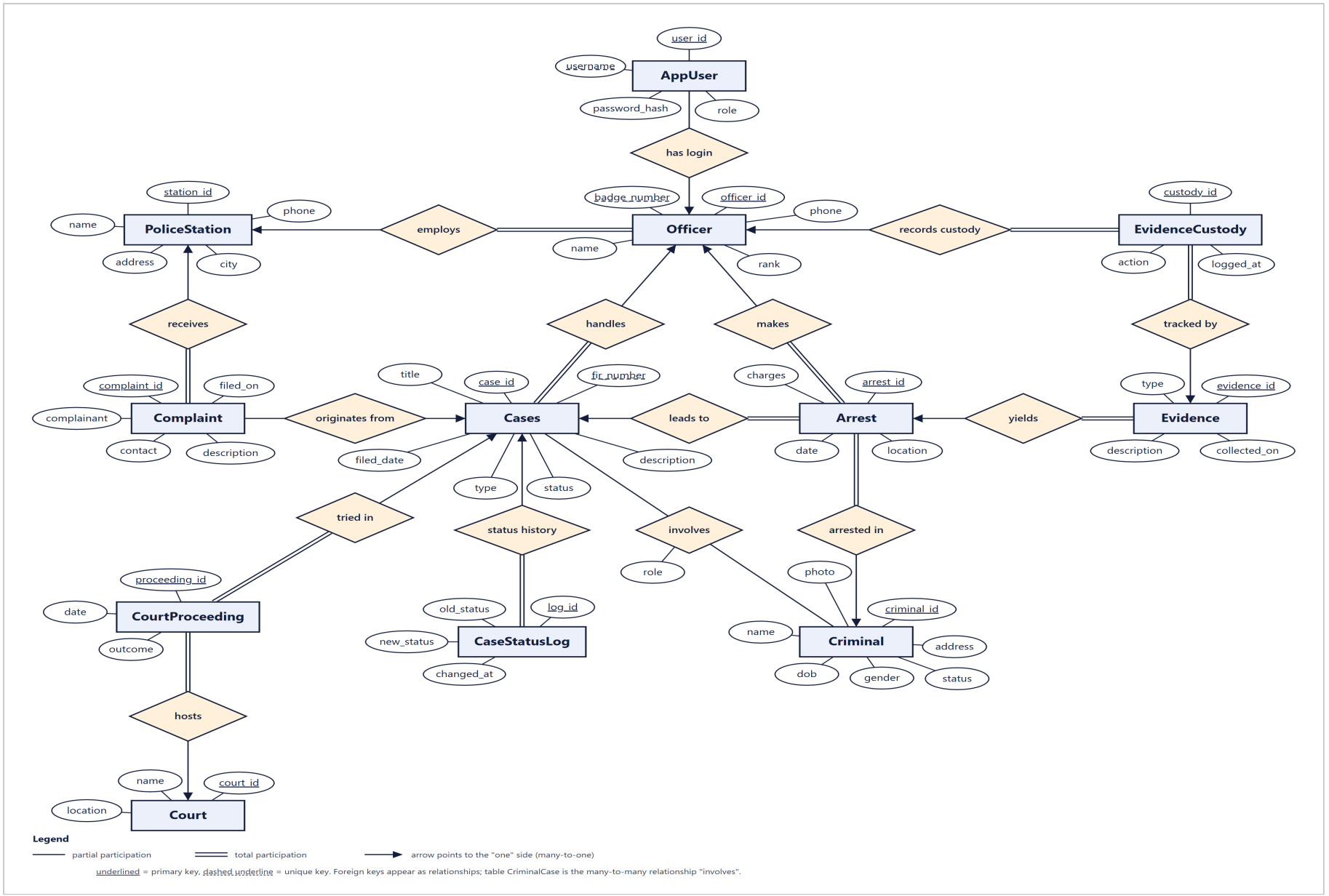


Figure 9: Entity-relationship diagram of the PCRMS database (Chen notation)

Reading the diagram: a police station employs many officers and receives many complaints; an officer handles cases, makes arrests and records custody actions, and may have login accounts (AppUser); a complaint may originate a case; an arrest leads to a case, arrests a criminal and yields evidence; each evidence item is tracked by custody records; a case is tried in court proceedings, which are hosted by a court, and accumulates status-history rows. The only many-to-many relationship is **involves** between Criminal and Cases. It is implemented by the junction table **CriminalCase** (composite primary key criminal_id, case_id), which is why the relational schema has 13 tables while the diagram shows 12 entities: the thirteenth table is the relationship itself, and its descriptive attribute **role** is attached to the diamond.

4.4 Relationships and Cardinalities

Table 8: Relationships between entities

Parent (one)	Child (many)	Foreign key	Cardinality	Optional?
PoliceStation	Officer	Officer.station_id	1 : N	No (NOT NULL)
PoliceStation	Complaint	Complaint.station_id	1 : N	No
Officer	Cases	Cases.officer_id	1 : N	No
Officer	Arrest	Arrest.officer_id	1 : N	No
Officer	EvidenceCustody	EvidenceCustody.officer_id	1 : N	No
Officer	AppUser	AppUser.officer_id	0..1 : N	Yes (NULL allowed)
Cases	Complaint	Complaint.case_id	0..1 : N	Yes (NULL allowed)
Cases	Arrest	Arrest.case_id	1 : N	No
Cases	CourtProceeding	CourtProceeding.case_id	1 : N	No
Cases	CaseStatusLog	CaseStatusLog.case_id	1 : N	No
Criminal	Arrest	Arrest.criminal_id	1 : N	No
Criminal and Cases	CriminalCase	criminal_id, case_id (composite PK)	M : N	Junction table
Arrest	Evidence	Evidence.arrest_id	1 : N	No
Evidence	EvidenceCustody	EvidenceCustody.evidence_id	1 : N	No
Court	CourtProceeding	CourtProceeding.court_id	1 : N	No

The database declares 16 foreign-key constraints (the 14 one-to-many relationships above plus the two foreign keys of the junction table). Appendix B lists each one with its ON DELETE and ON UPDATE rule.

4.5 Table Schemas

The following tables list every column of every table as read from the database: data type, whether NULL is allowed, default value and constraints. A dash means "none".

4.5.1 PoliceStation

Police stations with address, city and phone. Parent of officers and complaints.

Table 9: Schema of table PoliceStation

Column	Type	Null	Default	Constraints / notes
station_id	INT	No	-	Primary key; Auto-increment
name	VARCHAR(100)	No	-	-
address	VARCHAR(255)	No	-	-
city	VARCHAR(80)	No	-	-
phone	VARCHAR(20)	Yes	-	-

4.5.2 Officer

Police officers with unique badge number, rank and the station they belong to.

Table 10: Schema of table Officer

Column	Type	Null	Default	Constraints / notes
officer_id	INT	No	-	Primary key; Auto-increment
badge_number	VARCHAR(20)	No	-	Unique
name	VARCHAR(100)	No	-	-
rank	ENUM('Constable', 'Head Constable', 'Sub-Inspector', 'Inspector', 'DSP', 'SP')	No	-	-
phone	VARCHAR(20)	Yes	-	-
station_id	INT	No	-	FK to PoliceStation.station_id (ON DELETE NO ACTION, ON UPDATE CASCADE)

4.5.3 Criminal

Persons of interest with personal details, photo link and a status (Wanted, Arrested, Convicted, Released).

Table 11: Schema of table Criminal

Column	Type	Null	Default	Constraints / notes
criminal_id	INT	No	-	Primary key; Auto-increment
name	VARCHAR(100)	No	-	-
dob	DATE	Yes	-	-
gender	ENUM('Male', 'Female', 'Other')	Yes	-	-
address	VARCHAR(255)	Yes	-	-
status	ENUM('Wanted', 'Arrested', 'Convicted', 'Released')	No	Wanted	-

Column	Type	Null	Default	Constraints / notes
photo	VARCHAR(255)	Yes	-	-

4.5.4 Cases

Cases created from an FIR. Holds FIR number, type, status and the officer in charge. Named Cases because CASE is a reserved SQL word.

Table 12: Schema of table Cases

Column	Type	Null	Default	Constraints / notes
case_id	INT	No	-	Primary key; Auto-increment
fir_number	VARCHAR(30)	No	-	Unique
title	VARCHAR(150)	No	-	-
type	VARCHAR(60)	No	-	-
description	TEXT	Yes	-	-
filed_date	DATE	No	-	-
status	ENUM('Open', 'Under Investigation', 'In Court', 'Closed')	No	Open	-
officer_id	INT	No	-	FK to Officer.officer_id (ON DELETE NO ACTION, ON UPDATE CASCADE)

4.5.5 Complaint

Citizen complaints received at a station; optionally linked to the case they led to.

Table 13: Schema of table Complaint

Column	Type	Null	Default	Constraints / notes
complaint_id	INT	No	-	Primary key; Auto-increment
complainant	VARCHAR(100)	No	-	-
contact	VARCHAR(40)	Yes	-	-
description	TEXT	No	-	-
filed_on	DATE	No	-	-
station_id	INT	No	-	FK to PoliceStation.station_id (ON DELETE NO ACTION, ON UPDATE NO ACTION)
case_id	INT	Yes	-	FK to Cases.case_id (ON DELETE SET NULL, ON UPDATE NO ACTION)

4.5.6 CriminalCase

Junction table that resolves the many-to-many relationship between criminals and cases; stores the person's role in the case.

Table 14: Schema of table CriminalCase

Column	Type	Null	Default	Constraints / notes
criminal_id	INT	No	-	Primary key; FK to Criminal.criminal_id (ON DELETE CASCADE, ON UPDATE NO ACTION)
case_id	INT	No	-	Primary key; FK to Cases.case_id (ON DELETE CASCADE, ON UPDATE NO ACTION)
role	ENUM('Accused', 'Suspect', 'Convict')	No	Accused	-

4.5.7 Arrest

An arrest of a criminal in connection with a case by an officer, with date, location and charges.

Table 15: Schema of table Arrest

Column	Type	Null	Default	Constraints / notes
arrest_id	INT	No	-	Primary key; Auto-increment
date	DATE	No	-	-
location	VARCHAR(150)	Yes	-	-
charges	VARCHAR(255)	Yes	-	-
case_id	INT	No	-	FK to Cases.case_id (ON DELETE NO ACTION, ON UPDATE NO ACTION)
officer_id	INT	No	-	FK to Officer.officer_id (ON DELETE NO ACTION, ON UPDATE NO ACTION)
criminal_id	INT	No	-	FK to Criminal.criminal_id (ON DELETE NO ACTION, ON UPDATE NO ACTION)

4.5.8 Evidence

Evidence items collected for an arrest.

Table 16: Schema of table Evidence

Column	Type	Null	Default	Constraints / notes
evidence_id	INT	No	-	Primary key; Auto-increment
type	VARCHAR(60)	No	-	-
description	TEXT	Yes	-	-
collected_on	DATE	No	-	-
arrest_id	INT	No	-	FK to Arrest.arrest_id (ON DELETE CASCADE, ON UPDATE NO ACTION)

4.5.9 EvidenceCustody

Chain-of-custody log: each hand-over or action on an evidence item, with the officer and timestamp.

Table 17: Schema of table EvidenceCustody

Column	Type	Null	Default	Constraints / notes
custody_id	INT	No	-	Primary key; Auto-increment
evidence_id	INT	No	-	FK to Evidence.evidence_id (ON DELETE CASCADE, ON UPDATE NO ACTION)
officer_id	INT	No	-	FK to Officer.officer_id (ON DELETE NO ACTION, ON UPDATE NO ACTION)
action	VARCHAR(100)	No	-	-
logged_at	DATETIME	No	CURRENT_TIMESTAMP	-

4.5.10 Court

Courts with name and location.

Table 18: Schema of table Court

Column	Type	Null	Default	Constraints / notes
court_id	INT	No	-	Primary key; Auto-increment
name	VARCHAR(100)	No	-	-
location	VARCHAR(255)	No	-	-

4.5.11 CourtProceeding

Hearings of a case in a court, with date and outcome (NULL until concluded).

Table 19: Schema of table CourtProceeding

Column	Type	Null	Default	Constraints / notes
proceeding_id	INT	No	-	Primary key; Auto-increment
date	DATE	No	-	-
outcome	VARCHAR(255)	Yes	-	-
case_id	INT	No	-	FK to Cases.case_id (ON DELETE NO ACTION, ON UPDATE NO ACTION)
court_id	INT	No	-	FK to Court.court_id (ON DELETE NO ACTION, ON UPDATE NO ACTION)

4.5.12 AppUser

Application logins with bcrypt password hash and role (admin, officer, viewer); optionally linked to an officer.

Table 20: Schema of table AppUser

Column	Type	Null	Default	Constraints / notes
user_id	INT	No	-	Primary key; Auto-increment
username	VARCHAR(50)	No	-	Unique
password_hash	VARCHAR(100)	No	-	-
role	ENUM('admin', 'officer', 'viewer')	No	-	-
officer_id	INT	Yes	-	FK to Officer.officer_id (ON DELETE SET NULL, ON UPDATE NO ACTION)

4.5.13 CaseStatusLog

Audit rows written by a trigger whenever a case's status changes.

Table 21: Schema of table CaseStatusLog

Column	Type	Null	Default	Constraints / notes
log_id	INT	No	-	Primary key; Auto-increment
case_id	INT	No	-	FK to Cases.case_id (ON DELETE CASCADE, ON UPDATE NO ACTION)
old_status	VARCHAR(30)	No	-	-
new_status	VARCHAR(30)	No	-	-
changed_at	DATETIME	No	CURRENT_TIMESTAMP	-

4.6 Constraints and Indexes

Constraints are the first line of defence for data quality. Primary keys identify every row. NOT NULL prevents missing mandatory data. UNIQUE constraints protect natural keys that must not repeat: fir_number, badge_number, username and the pair (station name, city). ENUM types restrict status, rank, gender and role to a fixed vocabulary. Foreign keys prevent orphans and define what happens on deletion (NO ACTION, the MySQL default, behaves like RESTRICT in InnoDB: the delete is refused while child rows exist): for example, deleting a case cascades to its CriminalCase links and status log, but sets Complaint.case_id to NULL so the original citizen complaint is retained.

Table 22: Indexes in the pcrms database

Table	Index	Unique	Columns
AppUser	officer_id	No	officer_id
AppUser	username	Yes	username
Arrest	case_id	No	case_id
Arrest	idx_arrest_criminal	No	criminal_id
Arrest	officer_id	No	officer_id
Cases	fir_number	Yes	fir_number

Table	Index	Unique	Columns
Cases	idx_cases_status	No	status
Cases	officer_id	No	officer_id
CaseStatusLog	case_id	No	case_id
Complaint	case_id	No	case_id
Complaint	station_id	No	station_id
CourtProceeding	case_id	No	case_id
CourtProceeding	court_id	No	court_id
Criminal	idx_criminal_name	No	name
CriminalCase	case_id	No	case_id
Evidence	arrest_id	No	arrest_id
EvidenceCustody	evidence_id	No	evidence_id
EvidenceCustody	officer_id	No	officer_id
Officer	badge_number	Yes	badge_number
Officer	idx_officer_station	No	station_id
PoliceStation	name	Yes	name, city

Besides the explicit indexes idx_cases_status, idx_criminal_name, idx_officer_station and idx_arrest_criminal, MySQL creates an index for each unique constraint and for each foreign key column. Free-text search in the application uses LIKE with a leading wildcard, which cannot use an index and therefore scans the table; this is acceptable for the size of the demonstration data but is noted as a limitation in Chapter 8.

4.7 Normalization Analysis

The schema was normalized to third normal form (3NF) [14]. For each table the functional dependencies are those of its primary key; no non-key attribute determines another non-key attribute.

- **First normal form (1NF).** Every column holds a single atomic value and there are no repeating groups. Multiple criminals per case, multiple evidence items per arrest and multiple custody actions per evidence item are stored as rows in CriminalCase, Evidence and EvidenceCustody, not as lists in a column.
- **Second normal form (2NF).** Tables with a single-column primary key cannot have a partial dependency. In CriminalCase, whose key is (criminal_id, case_id), the only non-key attribute is role, which describes the pair and depends on the whole key.
- **Third normal form (3NF).** No transitive dependencies exist. Station details are stored only in PoliceStation and Officer holds only station_id; court details only in Court; a case stores officer_id and not the officer's name or rank; a custody row stores officer_id, not

the officer's details. Derived values such as the number of arrests per case are computed by views and queries and never stored.

Deliberate design choices that might look like redundancy: Arrest.case_id and Arrest.criminal_id both exist because an arrest identifies a person and the case it relates to, and CriminalCase additionally records the person's role in the case (the trigger trg_arrest_after_insert ensures the link exists). Cases.status is an explicit attribute rather than a derived value because it represents a workflow state that is changed by procedures and triggers and audited in CaseStatusLog.

4.8 Views, Stored Procedures and Triggers

Table 23: Programmability objects in the database

Object	Type	Purpose
v_case_overview	View	Each case with officer, station and counts of criminals, arrests and evidence items.
v_wanted_criminals	View	Criminals whose status is Wanted.
v_station_stats	View	Officers and cases handled per station.
sp_register_fir	Procedure	Creates a case inside a transaction and links an optional complaint; rolls back on any error.
sp_close_case	Procedure	Sets a case status to Closed.
sp_criminal_history	Procedure	Returns every case a criminal is linked to with the role and status.
trg_arrest_after_insert	Trigger (AFTER INSERT on Arrest)	Marks the criminal Arrested, moves an Open case to Under Investigation, links the criminal to the case.
trg_proceeding_after_insert	Trigger (AFTER INSERT on CourtProceeding)	Moves an Open or Under Investigation case to In Court.
trg_case_status_log	Trigger (AFTER UPDATE on Cases)	Writes a CaseStatusLog row when the status changes.
trg_evidence_custody_init	Trigger (AFTER INSERT on Evidence)	Creates the first EvidenceCustody row.

Listing 1: Stored procedure sp_register_fir (database/programmability.sql)

```
CREATE PROCEDURE sp_register_fir(IN p_fir VARCHAR(30), IN p_title VARCHAR(150), IN p_type VARCHAR(60),
  IN p_desc TEXT, IN p_officer INT, IN p_complaint INT)
BEGIN
  DECLARE new_id INT;
  DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
  START TRANSACTION;
  INSERT INTO Cases(fir_number,title,type,description,filed_date,officer_id)
    VALUES (p_fir,p_title,p_type,p_desc,CURDATE(),p_officer);
  SET new_id = LAST_INSERT_ID();
  IF p_complaint IS NOT NULL THEN
    UPDATE Complaint SET case_id=new_id WHERE complaint_id=p_complaint;
  END IF;
  COMMIT;
  SELECT new_id AS case_id;
END
```

The procedure above wraps the case insertion and complaint linking in one transaction. The exit handler rolls back and re-raises the error, which is why a duplicate FIR number leaves no partial data (verified by TC-25).

Listing 2: Trigger trg_arrest_after_insert (database/programmability.sql)

```
CREATE TRIGGER trg_arrest_after_insert AFTER INSERT ON Arrest FOR EACH ROW
BEGIN
  UPDATE Criminal SET status='Arrested' WHERE criminal_id=NEW.criminal_id;
  UPDATE Cases SET status='Under Investigation' WHERE case_id=NEW.case_id AND status='Open';
  INSERT IGNORE INTO CriminalCase(criminal_id,case_id) VALUES (NEW.criminal_id,NEW.case_id);
END
```

This trigger is the core lifecycle rule: one INSERT into Arrest updates three other tables consistently. Because it lives in the database, the rule applies no matter which client performs the insert.

4.9 How Data Is Stored, Retrieved, Updated and Deleted

- **Store (INSERT).** The API whitelists the columns of the target table, converts empty strings to NULL and runs one parameterized INSERT. Procedures and triggers perform secondary inserts and updates. Auto-increment surrogate keys are returned to the client.
- **Retrieve (SELECT).** List endpoints select from the table or from a join that adds display names (for example the officer name for a case). Detail pages combine several queries. Reports read from views or aggregate queries (COUNT, SUM, GROUP BY, HAVING). The criminal history page calls sp_criminal_history.
- **Update.** PUT requests update only the whitelisted columns that were sent. Status changes on Cases are captured by the audit trigger.
- **Delete.** Deletion is restricted to administrators. Foreign keys decide the outcome: a referenced row cannot be deleted (RESTRICT, reported to the user as a conflict), while child rows with ON DELETE CASCADE are removed with their parent and nullable references are set to NULL.

A representative query, the view that backs the case overview report, is shown below.

Listing 3: View v_case_overview (database/programmability.sql)

```
CREATE OR REPLACE VIEW v_case_overview AS
SELECT c.case_id, c.fir_number, c.title, c.type, c.status, c.filed_date,
       o.name AS officer, ps.name AS station,
       (SELECT COUNT(*) FROM CriminalCase cc WHERE cc.case_id=c.case_id) AS criminals,
       (SELECT COUNT(*) FROM Arrest a WHERE a.case_id=c.case_id) AS arrests,
       (SELECT COUNT(*) FROM Evidence e JOIN Arrest a ON a.arrest_id=e.arrest_id WHERE
a.case_id=c.case_id) AS evidence_items
FROM Cases c JOIN Officer o ON o.officer_id=c.officer_id
JOIN PoliceStation ps ON ps.station_id=o.station_id;
```

Chapter 5: System Implementation

5.1 Project Structure

The repository has three top-level folders: database (SQL), server (API) and client (user interface), plus the report folder. Table 24 lists the source files that make up the application; together they contain about 1684 lines.

Table 24: Main source files and their sizes

File	Lines	Purpose
client/src/App.jsx	102	Application shell: sidebar, top bar, routing, page transitions
client/src/pages/ResourcePage.jsx	174	Generic list, search, filter and add/edit/delete page used by nine modules
client/src/config.js	52	Field definitions that drive the generic page and navigation
client/src/pages/Dashboard.jsx	62	Statistics, charts and upcoming hearings
client/src/pages/CaseDetail.jsx	76	Case detail with linked criminals, hearings, evidence, history
client/src/pages/CriminalProfile.jsx	45	Criminal profile with case history and arrests
client/src/pages/Reports.jsx	57	Report selector, table, CSV export, print
client/src/pages/SearchResults.jsx	32	Global search results
client/src/pages/Login.jsx	72	Login screen
client/src/ui.jsx	81	Icons, page header, loading, empty and error states, badges
client/src/motion.jsx	34	Shared animation presets (Motion for React)
client/src/api.js	14	Fetch wrapper that attaches the JWT and handles 401
client/src/styles.css	241	Design tokens and all styling, responsive rules
server/index.js	174	Express app: auth, role middleware, dashboard, reports, special routes, generic CRUD
server/resources.js	25	Table configuration that generates the CRUD API
server/db.js	16	MySQL connection pool
server/setup.js	29	Creates the database, programmability objects, sample data and demo users
server/tests/run-tests.js	125	Automated API and database test runner (44 cases)
database/schema.sql	141	DDL for 13 tables, indexes and constraints
database/programmability.sql	76	Views, stored procedures and triggers
database/seed.sql	25	Sample data
database/queries.sql	31	Demonstration queries

5.2 Frontend Architecture

The frontend is a React 18 single-page application. main.jsx mounts the App component inside a BrowserRouter. **App.jsx** renders the shell (sidebar navigation, search box, user chip, logout) and the route table. Routes are generated from the resource configuration, so adding a module requires only a new entry in config.js and server/resources.js. Page transitions use AnimatePresence from Motion for React [4] keyed by the URL path.

The key design decision is the **generic ResourcePage**. Nine modules (stations, officers, criminals, cases, complaints, arrests, evidence, courts, proceedings) have the same shape: a searchable table plus an add/edit form. Instead of nine hand-written pages, one component reads a configuration object that lists each field, its label, whether it is required, whether it is a dropdown of enumerated values or a foreign key (loaded from the related endpoint) and which joined column to display. An excerpt for the Cases module follows.

Listing 4: Field configuration for the Cases module (client/src/config.js)

```
  cases: { title: 'Cases / FIR', pk: 'case_id', link: (r) => `/cases/${r.case_id}`, statusFilter:
['Open', 'Under Investigation', 'In Court', 'Closed'], fields: [
    { name: 'fir_number', label: 'FIR No.', required: true }, { name: 'title', label: 'Title',
required: true },
    { name: 'type', label: 'Type', required: true }, { name: 'filed_date', label: 'Filed On', type:
'date', required: true },
    { name: 'status', label: 'Status', options: ['Open', 'Under Investigation', 'In Court', 'Closed'],
required: true },
    { name: 'officer_id', label: 'Officer', ref: ref('officers', 'name'), required: true, show:
'officer' },
    { name: 'description', label: 'Description', type: 'textarea', hideInTable: true }] },
```

Specialized pages exist where the generic list is not enough: Dashboard (statistics and charts), CaseDetail (linked criminals with add/remove, hearings, evidence, status history, close case), CriminalProfile (case history and arrests), Reports (report selector with CSV export and print) and SearchResults. Shared presentation components (icons, page header, skeleton loader, empty state, error banner, status badge) are in ui.jsx.

Listing 5: API client (client/src/api.js)

```
export const getUser = () => JSON.parse(localStorage.getItem('user') || 'null');
export const logout = () => { localStorage.removeItem('token'); localStorage.removeItem('user'); };

export async function api(path, method = 'GET', body) {
  const res = await fetch('/api' + path, {
    method,
    headers: { 'Content-Type': 'application/json', Authorization: 'Bearer ' +
(localStorage.getItem('token') || '') },
    body: body ? JSON.stringify(body) : undefined,
  });
  const data = await res.json().catch(() => ({}));
  if (res.status === 401 && path !== '/login') { logout(); window.location.href = '/login'; }
  if (!res.ok) throw new Error(data.error || 'Request failed');
  return data;
}
```

Every request goes through this wrapper, which attaches the JWT from localStorage, parses JSON, logs the user out and redirects to the login page on a 401 response, and turns non-2xx responses into thrown errors whose message is displayed in an error banner.

5.3 Backend Architecture

The backend is a single Express application (server/index.js). Middleware order matters: JSON parsing and CORS first; the public POST /api/login route; then an authentication middleware that verifies the bearer token for every other /api route; then an authorization middleware that checks the role against the HTTP method; and finally the route handlers. Handlers are wrapped so that a rejected promise becomes a JSON error response.

The nine resources share one **generic CRUD router**. server/resources.js declares, for each resource, the table, the primary key, the writable columns, the searchable columns and an optional joined SELECT. A loop in index.js registers GET (list with search and status filter), GET by id, POST, PUT and DELETE for each resource, which yields 45 routes from a short loop.

Listing 6: Generic CRUD route generation, list and create handlers (server/index.js)

```
for (const [name, r] of Object.entries(resources)) {
  const base = `/api/${name}`;
  const pick = (body) => Object.fromEntries(r.fields.filter((f) => f in body).map((f) => [f, body[f]
=== '' ? null : body[f]]));
  const cols = (o) => Object.keys(o).map((k) => `\`${k}\``);

  app.get(base, wrap(async (req, res) => {
    const from = r.select ? `(${r.select}) t` : `\`${r.table}\` t`;
    const params = [];
    let where = '';
    if (req.query.q) {
      where = 'WHERE ' + r.search.map((c) => `t.\`${c}\` LIKE ?`).join(' OR ');
      r.search.forEach(() => params.push(`%${req.query.q}%`));
    }
    if (req.query.status) { where += (where ? ' AND ' : 'WHERE ') + 't.status = ?';
params.push(req.query.status); }
    const [rows] = await pool.query(`SELECT t.* FROM ${from} ${where} ORDER BY t.\`${r.pk}\` DESC`,
params);
    res.json(rows);
  }));
  app.get(`${base}/:id`, wrap(async (req, res) => {
    const [[row]] = await pool.query(`SELECT * FROM \`${r.table}\` WHERE \`${r.pk}\`=?`,
[req.params.id]);
    row ? res.json(row) : res.status(404).json({ error: 'Not found' });
  }));
  app.post(base, wrap(async (req, res) => {
    const d = pick(req.body);
    const [x] = await pool.query(`INSERT INTO \`${r.table}\` (${cols(d).join(',')}) VALUES
(${Object.keys(d).map(() => '?').join(',')})`, Object.values(d));
    res.status(201).json({ [r.pk]: x.insertId });
  }));
```

Because column names come only from the configuration (never from the request), the generic router is not vulnerable to column-name injection, and request values are always passed as query parameters. The pick() helper ignores any property that is not whitelisted,

which protects against mass assignment.

5.4 API Endpoints

PCRMS exposes 57 routes: the public login route, 11 special routes and the 45 generated CRUD routes. The complete reference is in Appendix A. The special routes are summarized in Table 25.

Table 25: Special (non-CRUD) API routes

Method and path	Purpose
POST /api/login	Verify credentials, return a JWT and the user (public).
GET /api/dashboard	Totals, cases by status and type, next five hearings.
GET /api/reports/:name	One of six reports (case-overview, wanted-criminals, station-stats, officer-workload, repeat-offenders, status-log).
POST /api/fir	Register an FIR by calling sp_register_fir.
POST /api/cases/:id/close	Close a case by calling sp_close_case.
GET /api/cases/:id/detail	Case, linked criminals, hearings, evidence and status history.
POST /api/cases/:id/criminals	Link a criminal to a case with a role.
DELETE /api/cases/:id/criminals/:cid	Remove a link.
GET /api/criminals/:id/history	Criminal profile, cases (sp_criminal_history) and arrests.
GET, POST /api/evidence/:id/custody	Read or append chain-of-custody entries.
GET /api/search?q=	Search criminals, cases and officers.

5.5 Database Integration

server/db.js creates one mysql2 promise pool (10 connections, dateStrings enabled so that DATE columns are returned as plain strings and not shifted by time zones). All SQL uses placeholders. Procedures are called with CALL statements and their result sets are read from the driver's result array. server/setup.js is run once (npm run setup-db): it executes schema.sql, splits programmability.sql on lines containing only $$ (the mysql2 driver does not support the DELIMITER command), loads seed.sql and inserts three demo users with bcrypt-hashed passwords.

5.6 Authentication and Authorization

A user logs in with a username and password. The server looks the user up, compares the password with the stored bcrypt hash and, on success, signs a JSON Web Token [9] containing the user id, username and role, valid for eight hours. The client stores the token in localStorage and sends it as a bearer token.

Listing 7: Login route (server/index.js)

```
app.post('/api/login', wrap(async (req, res) => {
  const { username, password } = req.body;
  const [[u]] = await pool.query('SELECT * FROM AppUser WHERE username=?', [username]);
  if (!u || !bcrypt.compareSync(password || '', u.password_hash)) return res.status(401).json({ error:
'Invalid credentials' });
  const user = { id: u.user_id, username: u.username, role: u.role };
  res.json({ token: jwt.sign(user, SECRET, { expiresIn: '8h' }), user });
}));
```

Listing 8: Role-based authorization middleware (server/index.js)

```
// viewer: read only; officer: no delete; admin: everything
const can = (role, method) => role === 'admin' || (role === 'officer' && method !== 'DELETE') ||
method === 'GET';
app.use('/api', (req, res, next) =>
  can(req.user.role, req.method) ? next() : res.status(403).json({ error: 'Your role is not allowed to
do this' }));
```

Table 26: Permissions by role

Role	GET (read)	POST / PUT (create, update)	DELETE
viewer	Allowed	Denied (403)	Denied (403)
officer	Allowed	Allowed	Denied (403)
admin	Allowed	Allowed	Allowed

The user interface mirrors these rules by hiding the Add and Edit buttons from viewers and the Delete button from everyone except administrators. The server-side check is the authoritative one; the hidden buttons are a convenience only.

5.7 Input Validation and Error Handling

Validation happens in layers. In the browser, required fields use the HTML required attribute and enumerated fields are dropdowns. In the API, only whitelisted columns are accepted and empty strings are converted to NULL. In the database, NOT NULL, ENUM, UNIQUE and FOREIGN KEY constraints are the final authority. The wrap() helper turns database errors into JSON responses, and the client shows the message in a red banner.

Listing 9: Error mapping (server/index.js)

```
const wrap = (fn) => (req, res) => fn(req, res).catch((e) => {
  const status = e.code === 'ER_DUP_ENTRY' ? 409 : e.errno === 1451 || e.errno === 1452 ? 409 : 500;
  res.status(status).json({ error: e.sqlMessage || e.message });
```

Table 27: Mapping of failures to HTTP responses

Situation	HTTP status	Where handled
Missing or invalid token	401	Authentication middleware; client redirects to login
Role not allowed	403	Authorization middleware
Wrong username or password	401	Login route
Duplicate unique value (for example FIR number)	409	wrap(): ER_DUP_ENTRY

Situation	HTTP status	Where handled
Foreign key violation (missing parent, or deleting a referenced row)	409	wrap(): MySQL errors 1451 and 1452
Unknown report or record	404	Report and by-id handlers
Other database errors, including missing required column and invalid ENUM value	500	wrap(): fallback. The text of the database error is returned.

The last row is a known weakness found during testing: invalid input is rejected correctly by the database but reported with status 500 rather than 400, and the raw MySQL message is returned. This is listed in Chapter 8 and Chapter 9.

5.8 Core Business Logic

FIR registration. The client posts the FIR to /api/fir, which calls sp_register_fir. The procedure sets the filed date to the current date, inserts the case with status Open (the column default) and updates the complaint's case_id, all in one transaction. The interface therefore hides the status and filed-date fields when adding a new case.

Status automation. Recording an arrest, scheduling a hearing and changing a case's status are handled by the triggers in Section 4.8; the application code does not duplicate these rules.

Listing 10: Audit trigger trg_case_status_log (database/programmability.sql)

```
CREATE TRIGGER trg_case_status_log AFTER UPDATE ON Cases FOR EACH ROW
BEGIN
  IF OLD.status <> NEW.status THEN
    INSERT INTO CaseStatusLog(case_id,old_status,new_status) VALUES
(NEW.case_id,OLD.status,NEW.status);
  END IF;
END
```

Chain of custody. When evidence is inserted, a trigger writes the first custody row using the officer who made the arrest. Further actions are appended through /api/evidence/:id/custody and are shown in the custody dialog in oldest-first order.

Reporting. The report endpoints are plain SQL: views for the case overview, wanted criminals and station statistics, and aggregate queries for officer workload, repeat offenders (HAVING COUNT > 1) and the status log. The client turns the returned rows into a table and can export them to CSV in the browser.

The save() function of the generic page shows how the interface chooses between update, FIR registration and ordinary insert:

Listing 11: Create/update dispatch in the generic page (client/src/pages/ResourcePage.jsx)

```
const save = async (form) => {
    try {
      if (form[cfg.pk]) await api(`/${name}/${form[cfg.pk]}`, 'PUT', form);
      else if (name === 'cases') await api('/fir', 'POST', form); // stored procedure sp_register_fir
      else await api(`/${name}`, 'POST', form);
      setEditing(null); setError(''); load();
    } catch (e) { setError(e.message); }
  };
```

5.9 Third-Party Libraries and Dependencies

Table 28: Runtime and build dependencies (versions from package-lock.json)

Package	Version	Used for
react, react-dom	18.3.1	User interface [1]
react-router-dom	6.30.6	Client-side routing [3]
motion	14.0.0	Animations (imports from motion/react) [4]
vite, @vitejs/plugin-react	5.4.21, 4.7.0	Development server, proxy and build [2]
express	4.22.3	REST API [6]
mysql2	3.24.5	MySQL driver and pool [8]
jsonwebtoken	9.0.3	Signing and verifying JWTs [10]
bcryptjs	2.4.3	Password hashing [11]
cors, dotenv	2.8.6, 16.6.1	Cross-origin headers; environment configuration

Mermaid [12] and Puppeteer Core [13] were used outside the project to render the diagrams and capture the screenshots of this report; they are not dependencies of the application.

5.10 Integration of Frontend, Backend and Database

In development the browser loads the UI from the Vite server on port 5173. Calls to /api are proxied to Express on port 4000, which uses the pool to reach MySQL on port 3306. A typical interaction, such as opening the Cases page, is: the page mounts, ResourcePage calls api("/cases"), the wrapper adds the bearer token, Express verifies it, checks the role, runs the joined SELECT with the search and status parameters and returns JSON, and React renders the rows. Foreign-key dropdowns issue additional GET requests to the related resources.

5.11 Important Implementation Decisions

Table 29: Key design decisions and trade-offs

Decision	Benefit	Trade-off
Rules in the database (procedures, triggers)	Consistent behaviour for any client; fewer places to maintain.	Logic is split between SQL and JavaScript; triggers are less visible to application developers.
Generic CRUD router and generic list page	Nine modules from two small configuration files; consistent behaviour.	Special behaviour needs explicit exceptions (for example the FIR route).

Decision	Benefit	Trade-off
Table renamed Cases	Avoids the reserved word CASE.	Name differs from the project review document.
Whitelisted columns and parameterized SQL	Prevents SQL injection and mass assignment.	Every new column must be added to the configuration.
JWT stored in localStorage	Simple stateless client.	Exposed to cross-site scripting if the UI were compromised; httpOnly cookies would be stronger.
No self-registration	Accounts for a police system are issued by administrators.	Account management needs SQL or a script; there is no admin screen.
Database errors passed through as messages	Users see the real reason (for example a foreign key conflict).	Exposes internal table and constraint names; validation errors return 500.

Chapter 6: User Interface and Project Screenshots

All screenshots in this chapter and Appendix C were captured from the running application by an automated browser (headless Chrome 154 via Puppeteer Core, desktop viewport 1440 x 900 at 1.5x scale, mobile viewport 390 x 844 at 2x scale) against the seeded sample database. They show real data, were not edited, and are the same pages that were exercised by the UI tests in Chapter 7. The application has no separate marketing landing page: the login screen is the entry point.

6.1 Login Screen

Figure 10 shows the login page. The left panel introduces the system; the right panel holds the form. The three demo accounts can be filled in with one click, which is useful for demonstration and is stated on screen. The page verifies requirement FR-01.

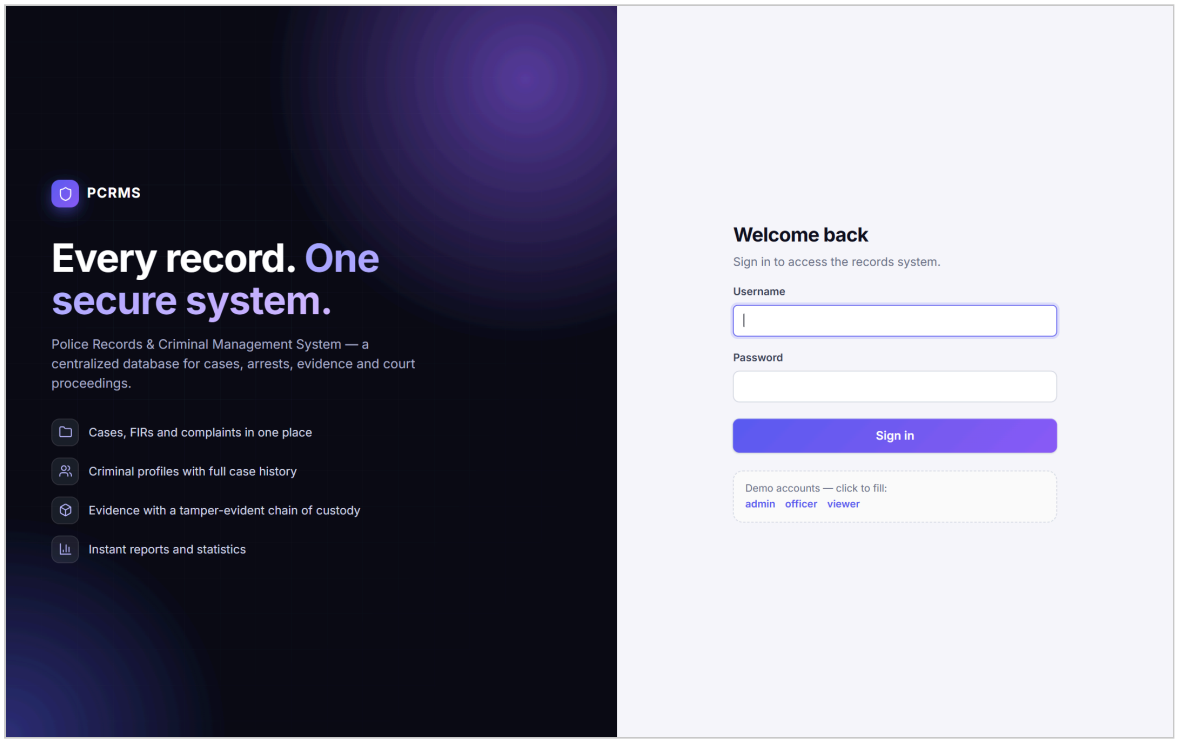


Figure 10: Login screen

Figure 11 shows the error state after a wrong password: the server answers 401 and the interface displays "Invalid credentials" without leaving the page (UI-02).

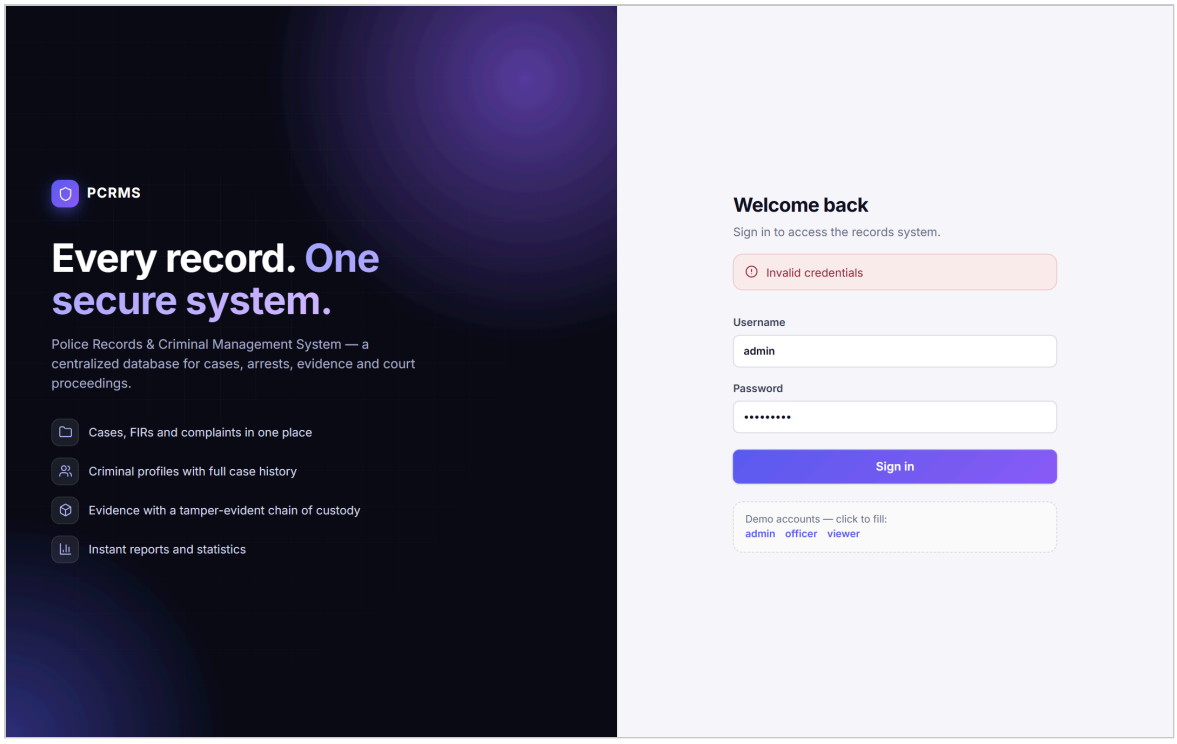


Figure 11: Login error state after an incorrect password

6.2 Dashboard

Figure 12 is the dashboard shown after login. It displays eight totals, bar charts of cases by status and by type, and the next hearings, all computed live from the database through GET /api/dashboard. It verifies FR-14.

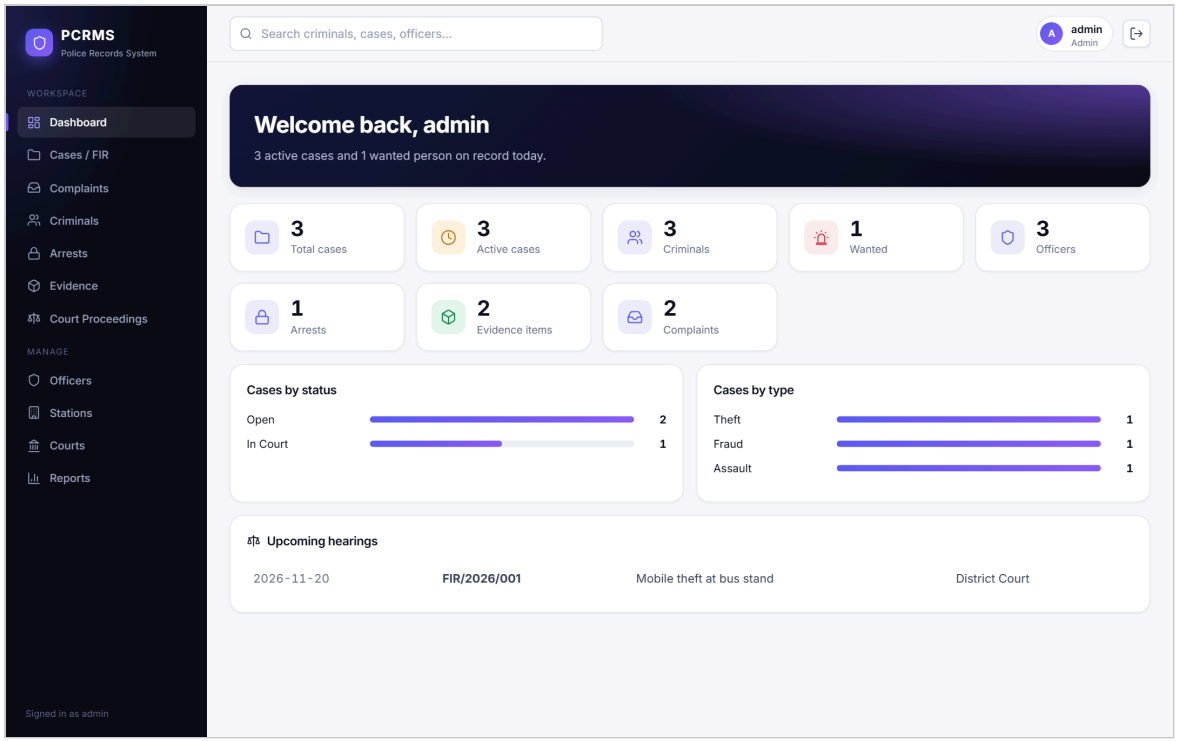


Figure 12: Dashboard with statistics, charts and upcoming hearings

6.3 Lists, Search and Filtering

Figure 13 shows the Cases module, which is representative of all nine list pages: a header with the record count, a search box, a status filter (for cases), an "Add new" button and a table with colour-coded status badges and edit and delete buttons. Rows come from the joined query that adds the officer name.

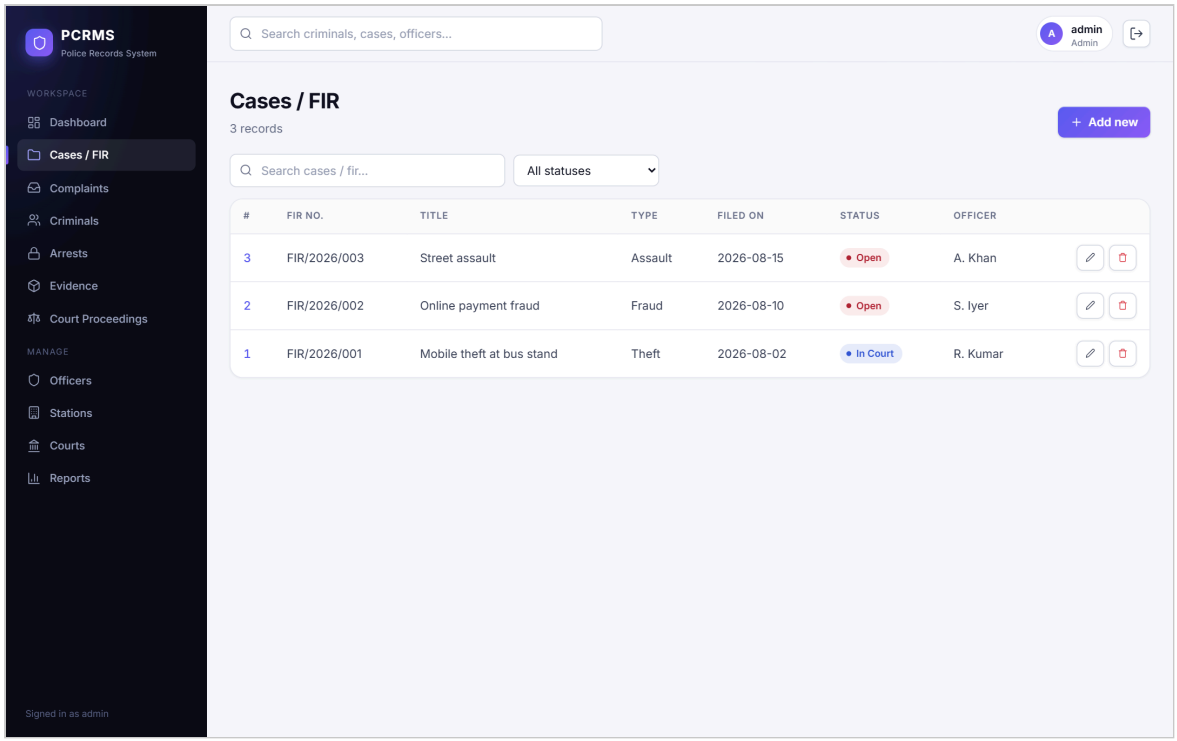


Figure 13: Cases / FIR list

Figure 14 shows the status filter set to In Court: only the matching case remains (UI-18).

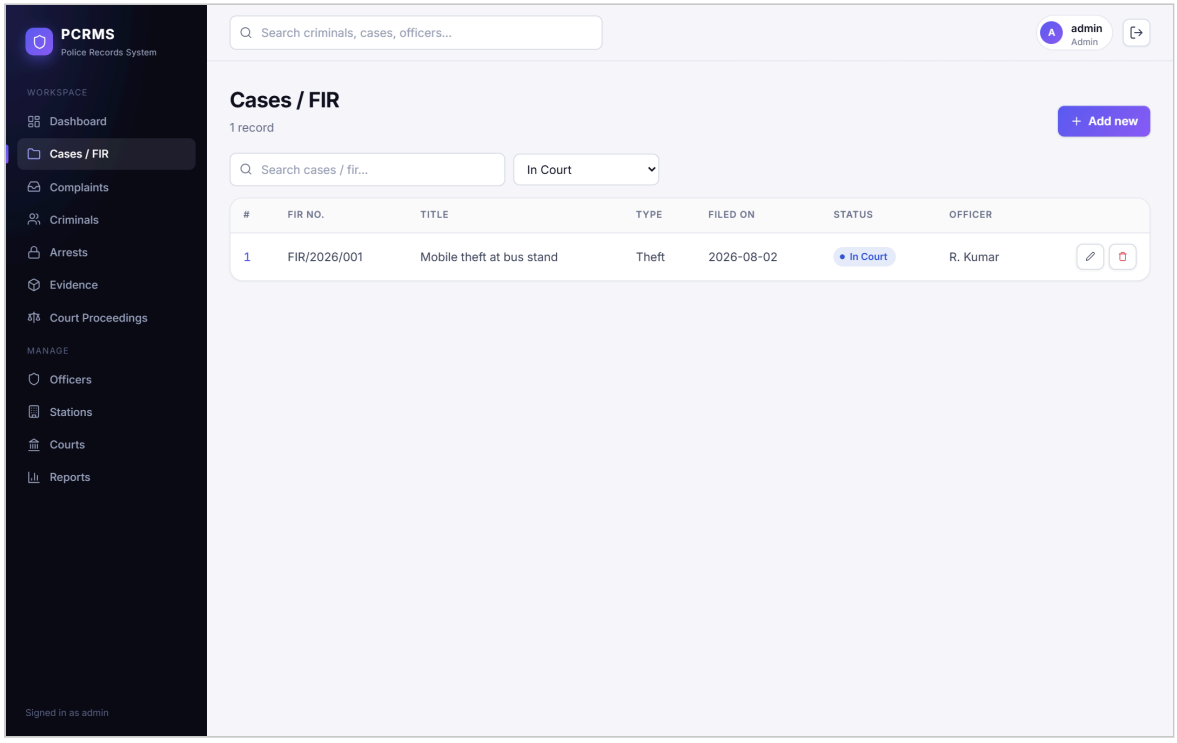


Figure 14: Cases filtered by status "In Court"

Figure 15 shows the global search. Typing "theft" in the top bar and pressing Enter lists matching criminals, cases and officers with links to the detail pages (UI-21).

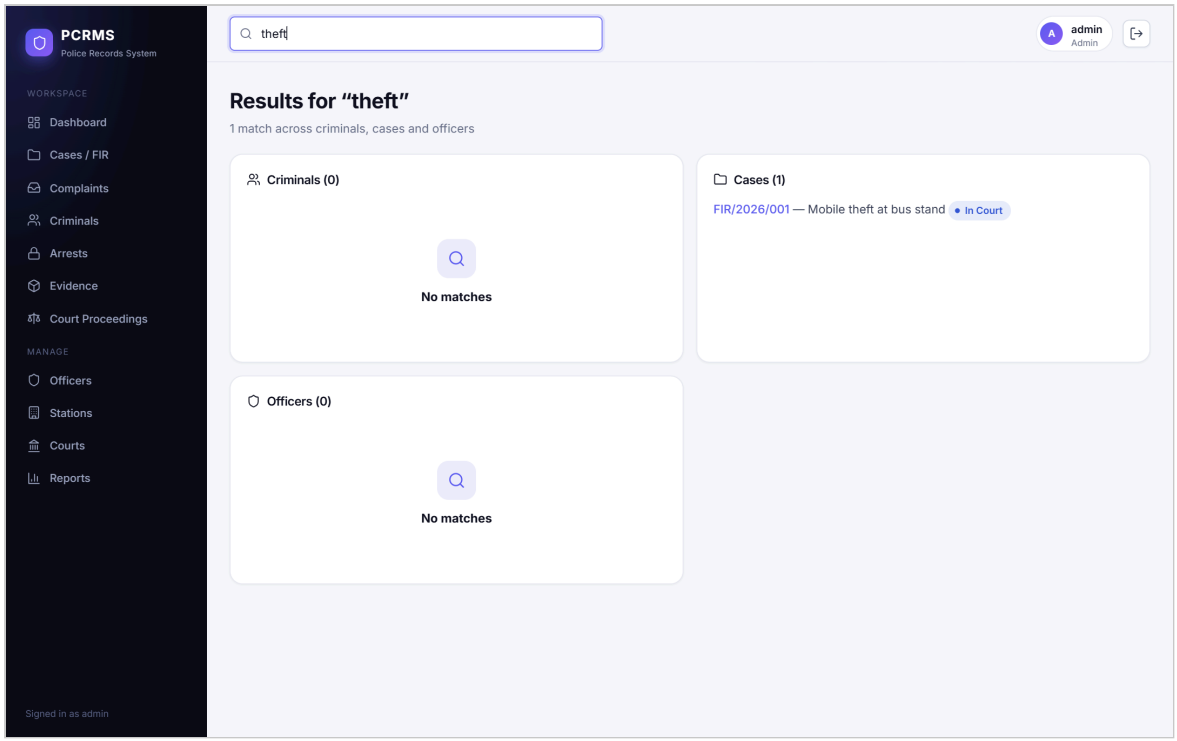


Figure 15: Global search results

6.4 Detail Pages

Figure 16 shows the case detail page: the case header with status badge, the officer and station, the linked criminals with a form to link another person and a role, the court proceedings, the evidence and the status history written by the trigger. It combines data from six tables.

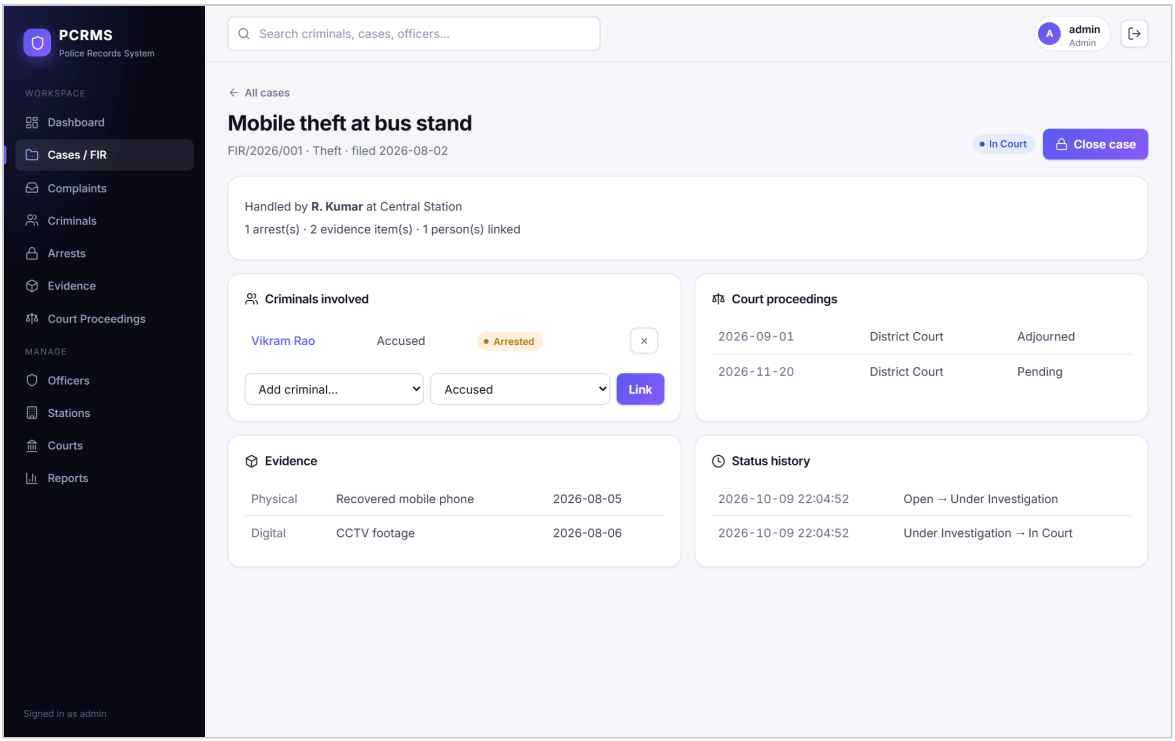


Figure 16: Case detail page

Figure 17 shows a criminal profile with personal details, case history and arrests (data from sp_criminal_history and the Arrest table).

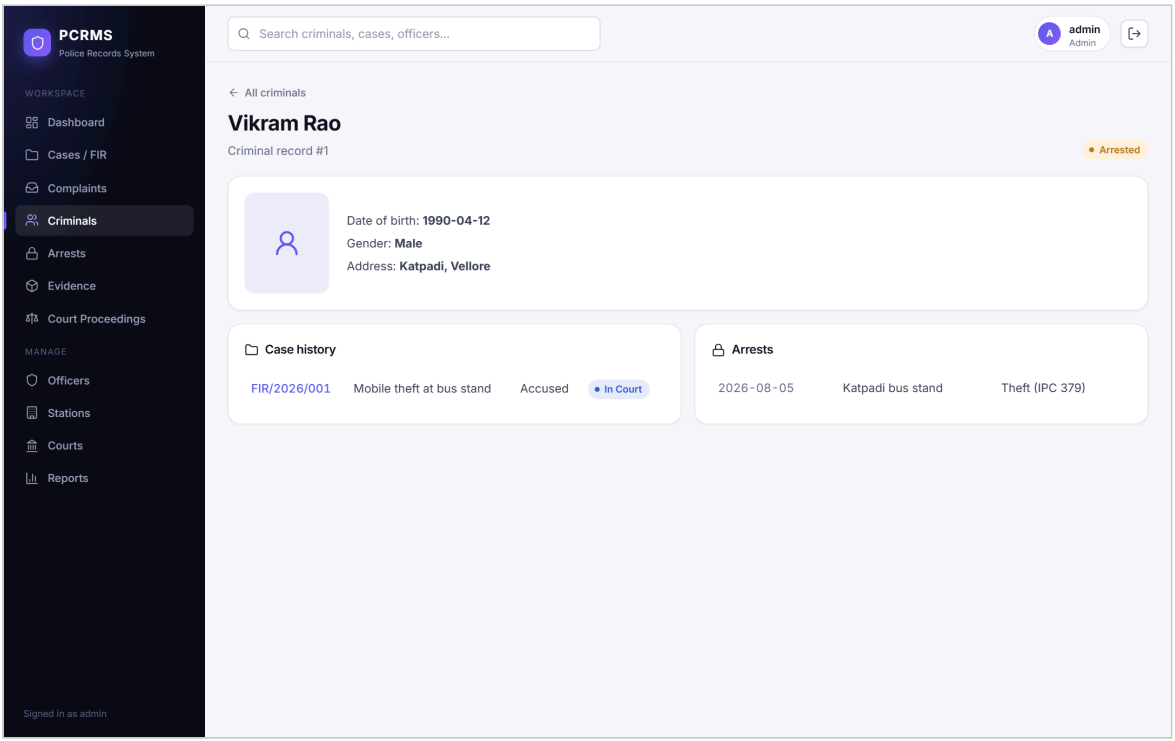


Figure 17: Criminal profile page

Figure 18 shows the Criminals list, including the status badges (Wanted, Arrested, Convicted).

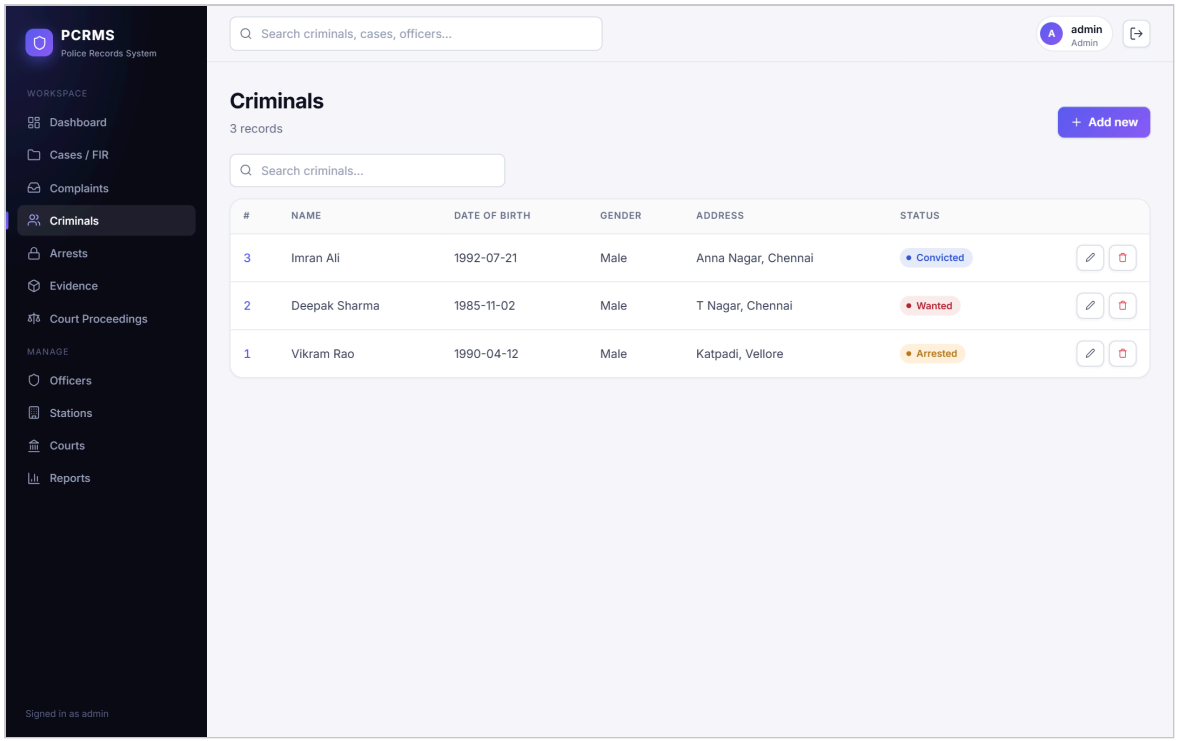


Figure 18: Criminals list

6.5 Forms and Data Entry

Figure 19 shows the add form for an arrest. The Criminal, Case and Officer fields are dropdowns filled from the database, which enforces valid foreign keys at entry time; required fields are marked with an asterisk. Saving posts to /api/arrests and fires the arrest trigger (UI-16).

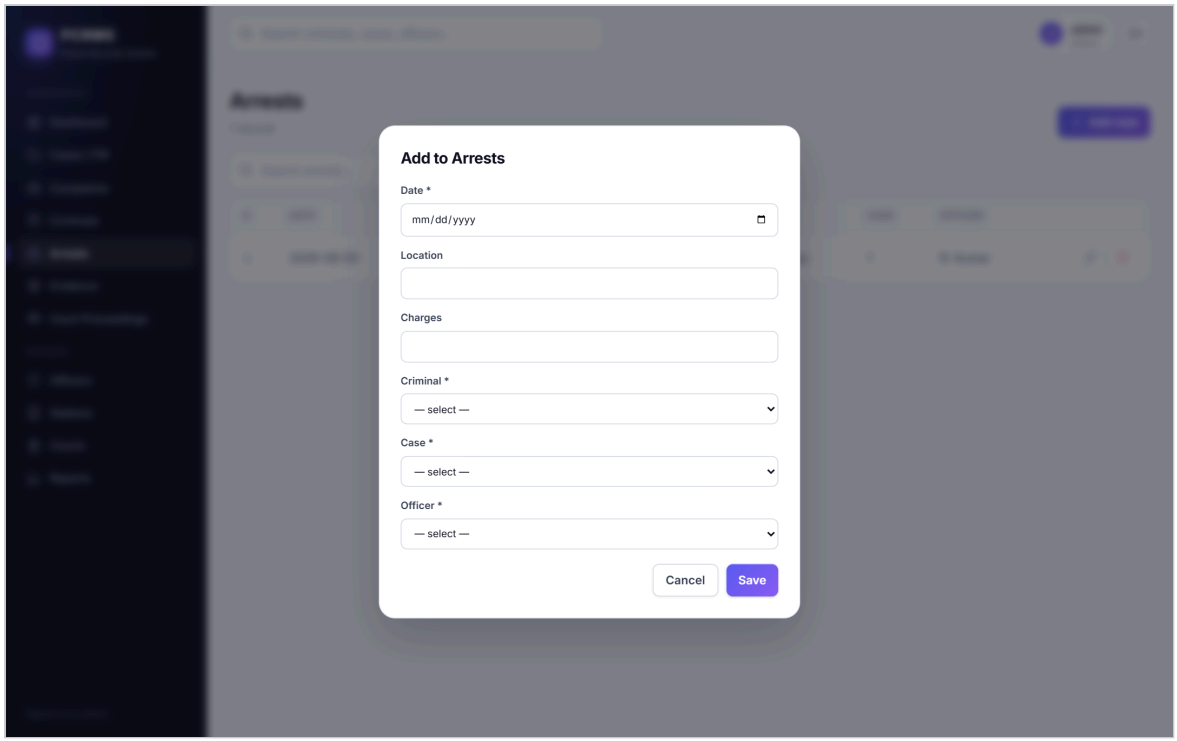


Figure 19: Add-arrest form with foreign-key dropdowns

Figure 20 shows the chain-of-custody dialog for an evidence item. The first entry was created automatically by the evidence trigger; officers can append further actions.

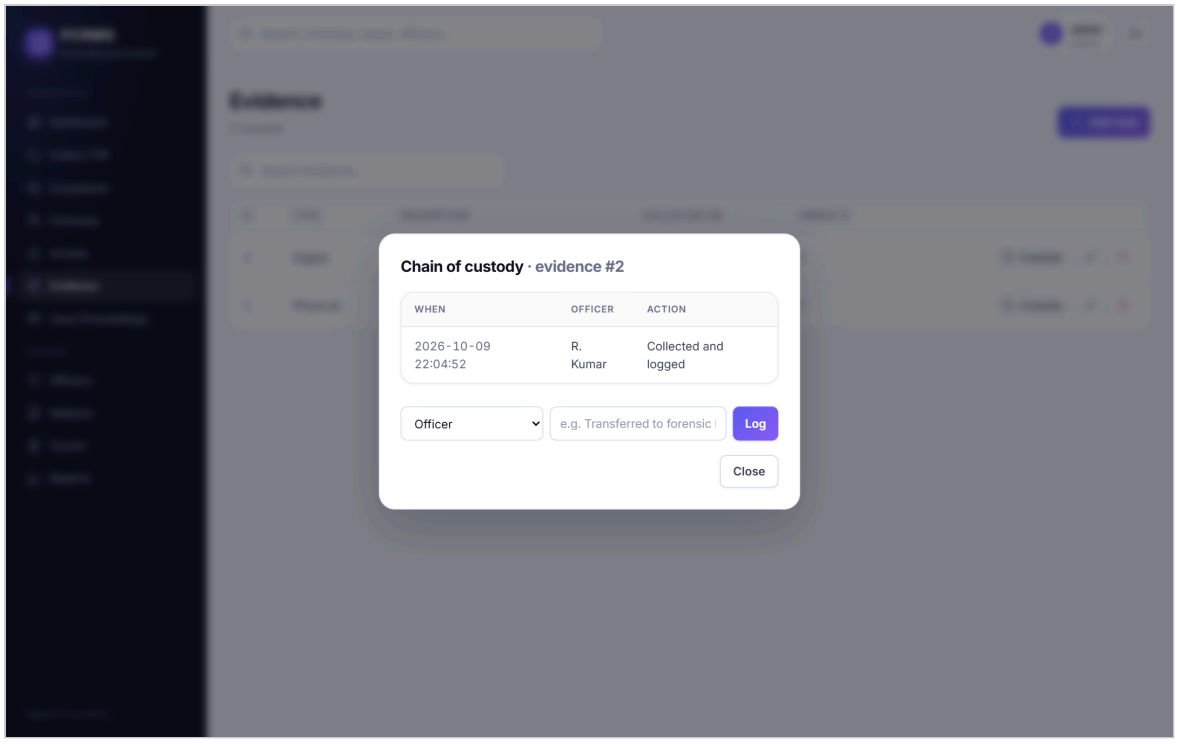


Figure 20: Chain-of-custody dialog

6.6 Reports

Figure 21 shows the Reports page with the case-overview report (data from the view v_case_overview) and the Export CSV and Print buttons (UI-20). Figure 22 shows another report, officer workload, produced by an aggregate query.

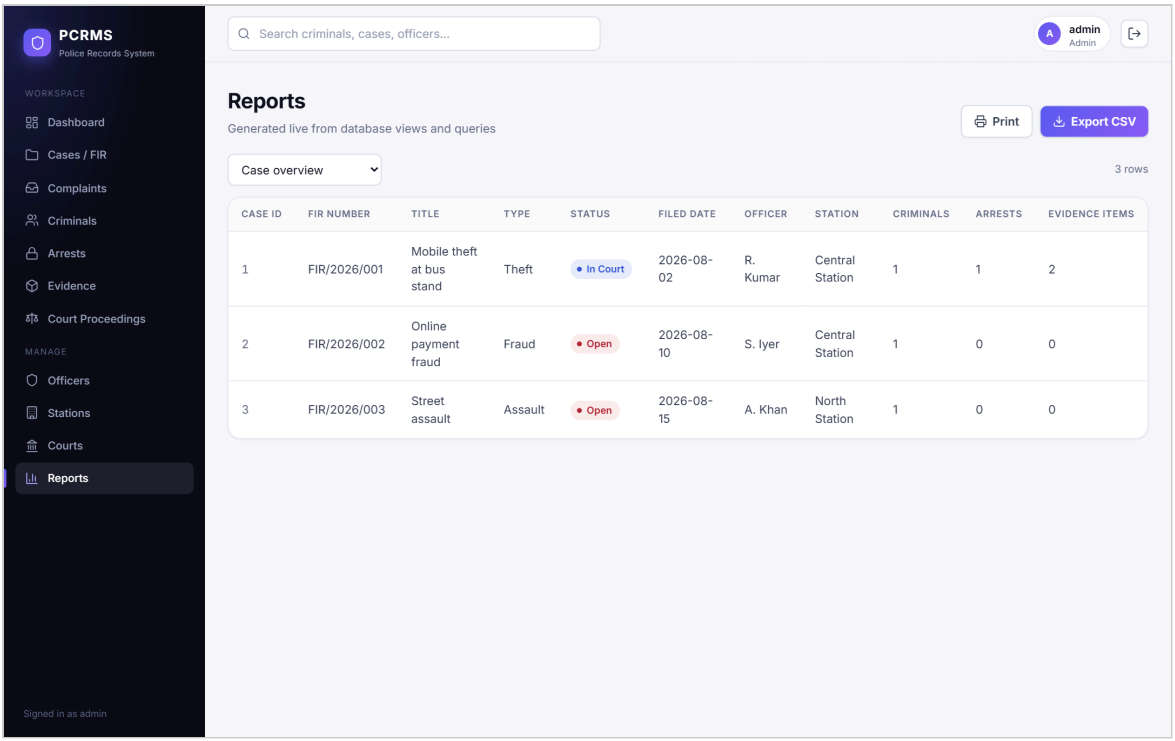


Figure 21: Reports page: case overview

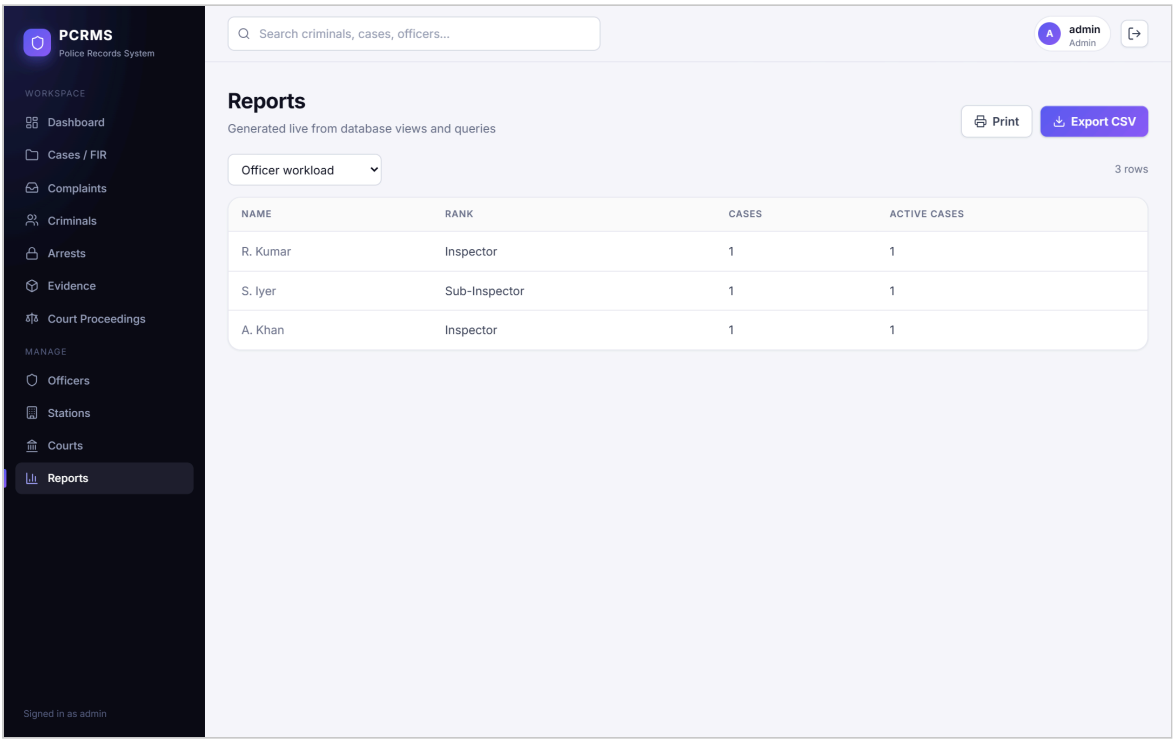


Figure 22: Reports page: officer workload

6.7 Empty and Error States

Figure 23 shows the empty state for a search with no matches. Figure 24 shows a database error surfaced to the user: an administrator tried to delete an officer who still handles cases,

and the foreign-key constraint blocked it (UI-22; API test TC-22). The record remains.

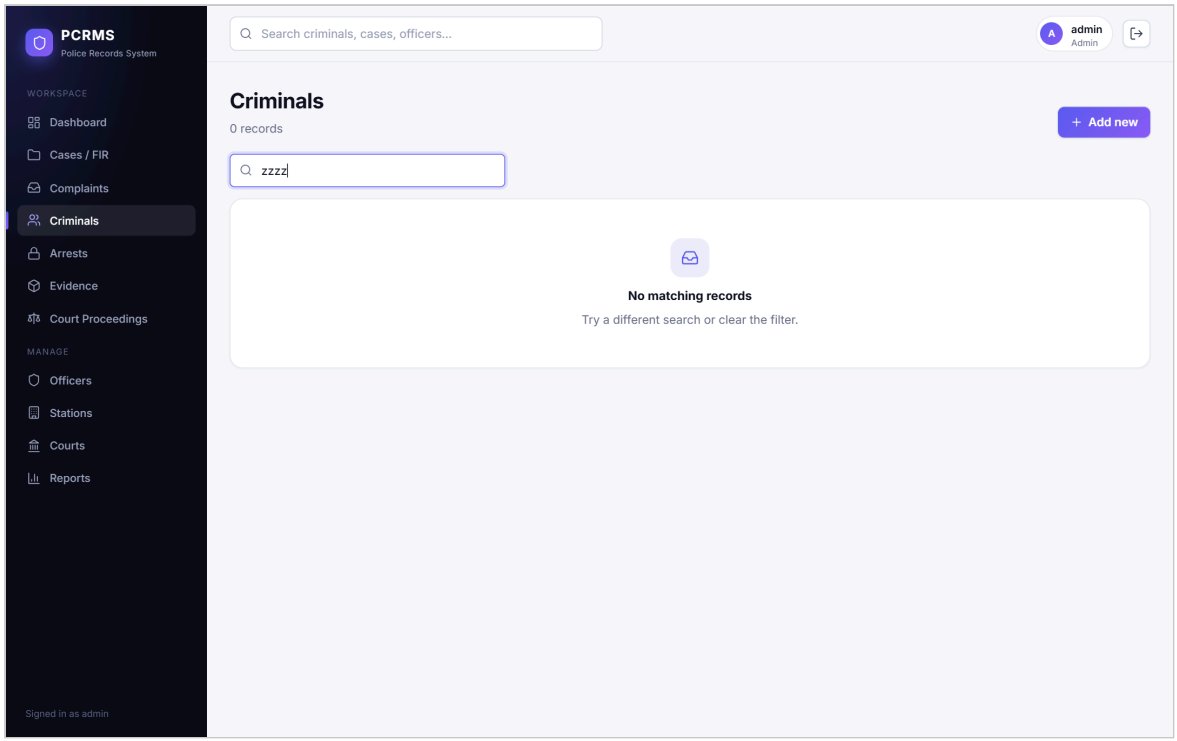


Figure 23: Empty state when no record matches

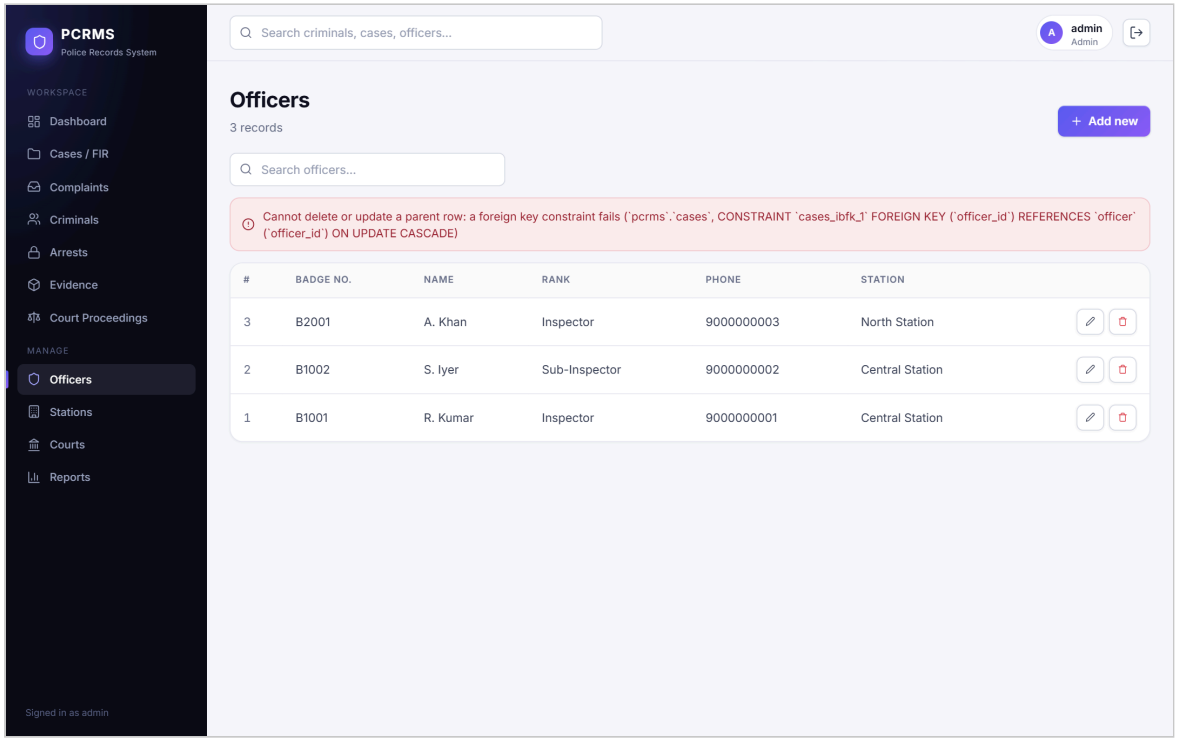


Figure 24: Database constraint error shown in the interface

6.8 Role-Based Interface

Figure 25 shows the Cases page for the viewer role: the Add button and the Edit and Delete

buttons are not rendered (UI-23). The same restrictions are enforced by the API (403).

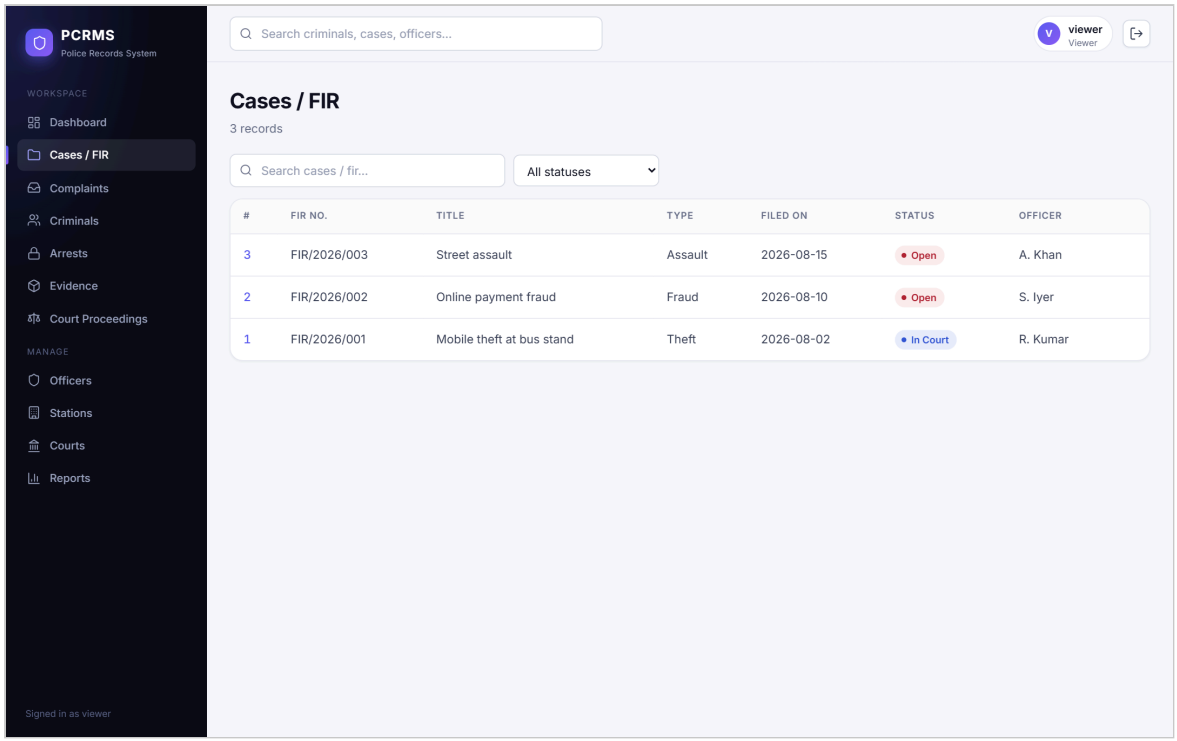


Figure 25: Cases page as seen by the read-only viewer role

6.9 Responsive and Mobile Layout

Below 900 px the sidebar becomes a slide-in drawer opened with a menu button, tables scroll horizontally inside their card, and controls stack vertically. Figure 26, Figure 27 and Figure 28 were captured at 390 x 844 (UI-24 confirmed that the page has no horizontal overflow and that the drawer opens).

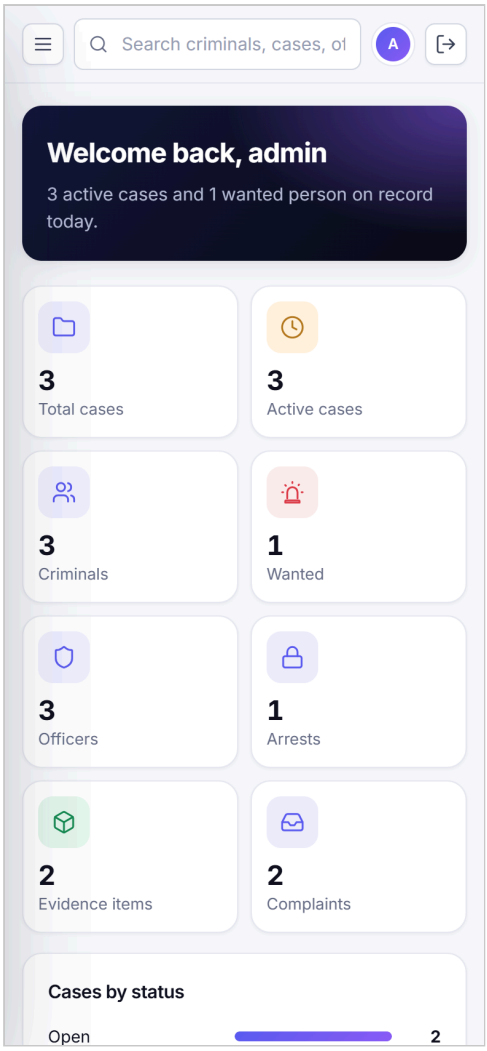


Figure 26: Dashboard on a 390 px wide screen

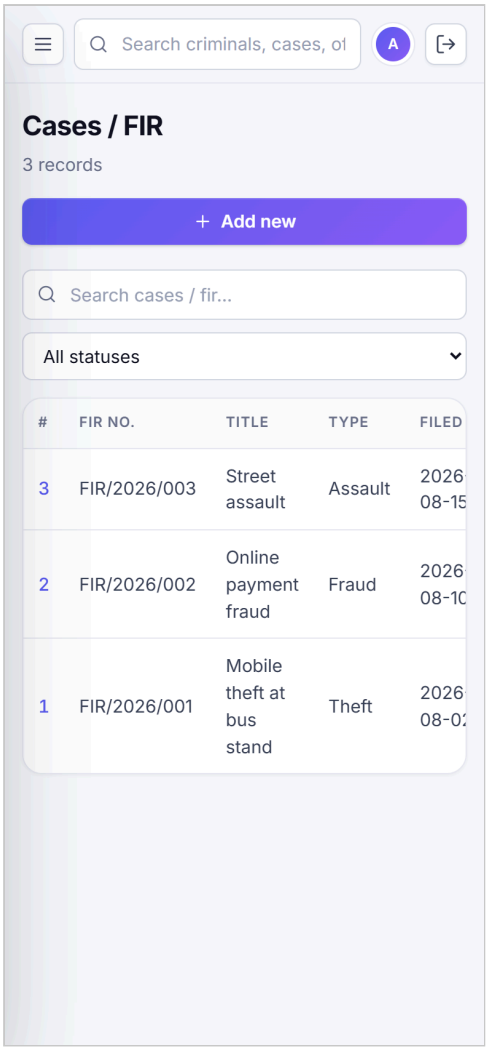


Figure 27: Cases list on a 390 px wide screen

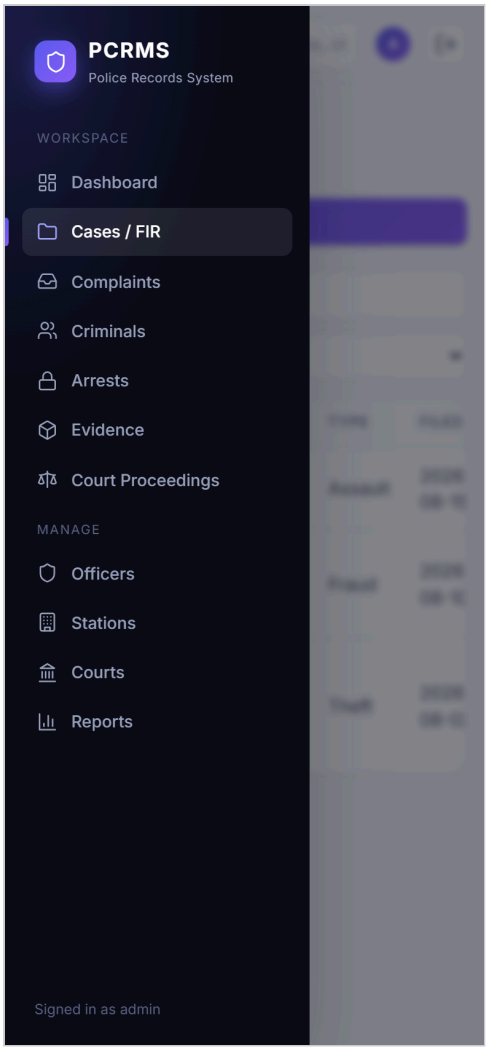


Figure 28: Navigation drawer on a 390 px wide screen

6.10 Screens That Are Not Available

- **Registration screen:** not implemented by design (see Section 2.7); there is therefore no screenshot.
- **Success messages:** the application does not show toast or success banners. After a save the dialog closes and the list refreshes; the new row is the confirmation. No success-message screenshot exists.
- **Browser-native validation bubbles** (shown when a required field is empty) are drawn by the browser outside the page content and could not be captured by the automated screenshot method.

Chapter 7: Testing and Validation

7.1 Testing Strategy

Testing focused on the layers where defects would be most damaging: the database rules, the API and the user interface. Three kinds of executed test are reported:

- **API and database integration tests** (44 cases, server/tests/run-tests.js). A Node script logs in as each role and calls the live API, then inspects the database directly with SQL to confirm what was stored. This covers authentication, authorization, CRUD, validation, referential integrity, stored procedures, triggers and reports.
- **User-interface (system) tests** (24 cases). An automated headless Chrome session drives the real React application against the real API and database: it logs in, navigates every page, opens forms, filters, searches, triggers an error and checks the page content, and resizes to a phone viewport.
- **Negative and security tests** are included in both sets: wrong credentials, missing and tampered tokens, forbidden roles, invalid enum values, missing required fields, duplicate keys, foreign-key violations and an SQL-injection attempt.

The database was reset to the seed data (npm run setup-db) before each run so that test data was deterministic.

7.2 Test Environment

Windows 11 Home; Node.js v22.19.0; MySQL Community Server 8.4.9 running locally; API on port 4000; Vite development server on port 5173; Google Chrome 154.0.8037.98 driven by Puppeteer Core. Executed on 2026-10-09.

7.3 API and Database Test Cases

Table 30 lists all 44 API and database test cases with the result actually observed. Each row's "Actual result" column is written by the test runner at execution time, not by hand. 44 of 44 passed.

Table 30: API and database test cases and results (all executed)

ID	Feature: test	Preconditions	Input	Expected result	Actual result	Status
TC-01	Authentication: Valid login	Seeded user admin exists	admin / correct password	HTTP 200, JWT token and user role returned	HTTP 200, token issued, role=admin	Pass
TC-02	Authentication: Wrong password rejected	Seeded user admin exists	admin / wrong password	HTTP 401, no token	HTTP 401 "Invalid credentials"	Pass
TC-03	Authentication: Unknown user rejected	None	unknown user / any password	HTTP 401	HTTP 401	Pass

ID	Feature: test	Preconditions	Input	Expected result	Actual result	Status
TC-04	Authentication: Protected route without token	None	GET /api/cases with no Authorization header	HTTP 401	HTTP 401 "Login required"	Pass
TC-05	Authentication: Tampered token rejected	None	GET /api/cases with an invalid JWT	HTTP 401	HTTP 401	Pass
TC-06	Authentication: Password hash not stored in plain text	Database seeded	SELECT password_hash FROM AppUser	Values are bcrypt hashes, not the plain passwords	3 users, all hashes start with $2 (bcrypt)	Pass
TC-07	Authorization: Viewer can read	Logged in as viewer	GET /api/cases	HTTP 200 with rows	HTTP 200, 3 rows	Pass
TC-08	Authorization: Viewer cannot create	Logged in as viewer	POST /api/courts	HTTP 403	HTTP 403 "Your role is not allowed"	Pass
TC-09	Authorization: Officer can create	Logged in as officer	POST /api/courts {name, location}	HTTP 201 with new id	HTTP 201, court_id=3	Pass
TC-10	Authorization: Officer cannot delete	Logged in as officer; court created above	DELETE /api/courts/:id	HTTP 403, record remains	HTTP 403, record still present	Pass
TC-11	Authorization: Admin can delete	Logged in as admin; court created above	DELETE /api/courts/:id	HTTP 200, record removed	HTTP 200, record removed	Pass
TC-12	CRUD: Create station	Admin token	POST /api/stations with valid body	HTTP 201, row inserted	HTTP 201, station_id=3	Pass
TC-13	CRUD: Read station by id	Station created	GET /api/stations/:id	HTTP 200 with the same values	HTTP 200, city=Testville	Pass
TC-14	CRUD: Update station	Station created	PUT /api/stations/:id {city: "Newtown"}	HTTP 200, value changed in DB	HTTP 200, DB city=Newtown	Pass
TC-15	CRUD: Search filter	Seed data	GET /api/criminals? q=Vikram	Only matching rows returned	1 row: Vikram Rao	Pass
TC-16	CRUD: Status filter	Seed data	GET /api/cases? status=Closed on seed data	Empty list (no closed cases yet)	HTTP 200, 0 rows	Pass
TC-17	CRUD: Delete station	Station created	DELETE /api/stations/:id	HTTP 200, row removed	HTTP 200, row removed	Pass
TC-18	Validation: Missing required column	Admin token	POST /api/stations with only {name}	Rejected with an error, nothing inserted	HTTP 500, row count unchanged (2)	Pass
TC-19	Validation: Invalid ENUM value	Admin token	PUT /api/criminals/1 {status: "Flying"}	Rejected, status unchanged	HTTP 500, status still Arrested	Pass
TC-20	Validation: Duplicate unique FIR number	FIR/2026/001 exists	POST /api/cases with fir_number FIR/2026/001	HTTP 409 duplicate	HTTP 409 (duplicate entry)	Pass
TC-21	Validation: Foreign key must exist	Admin token	POST /api/officers with station_id 9999	HTTP 409 FK violation	HTTP 409 (foreign key constraint fails)	Pass
TC-22	Validation: Cannot delete referenced parent row	Officer 1 handles cases	DELETE /api/officers/1	HTTP 409, officer remains	HTTP 409, officer still present	Pass
TC-23	Validation: SQL injection attempt in search	Seed data	GET /api/search?q=' OR 1=1; DROP TABLE Criminal;--	No error, tables intact (parameterized queries)	HTTP 200, Criminal table intact (3 rows)	Pass
TC-24	Stored procedure: sp_register_fir with complaint	Seeded complaint 1 exists	POST /api/fir {FIR/2026/090, complaint_id 1}	Case created with status Open and today as filed date; complaint linked to new case	HTTP 201, case_id=5, status=Open, complaint 1 now linked	Pass
TC-25	Stored procedure: sp_register_fir rolls back on error	FIR/2026/090 exists	POST /api/fir with duplicate FIR number	HTTP 409, no extra case row	HTTP 409, case count unchanged (4)	Pass

ID	Feature: test	Preconditions	Input	Expected result	Actual result	Status
TC-26	Trigger: Arrest trigger updates criminal and case	Criminal 2 Wanted; case 2 Open	POST /api/arrests {criminal 2, case 2}	Criminal status=Arrested, case status=Under Investigation	criminal 2 = Arrested, case 2 = Under Investigation	Pass
TC-27	Trigger: Arrest trigger links criminal to case	Arrest above created	Query CriminalCase for (2,2)	Row exists (already linked as Suspect)	1 row, role=Suspect (INSERT IGNORE kept the original)	Pass
TC-28	Trigger: Status change logged	Case 2 changed status above	Query CaseStatusLog for case 2	Row Open -> Under Investigation	Log row: Open -> Under Investigation	Pass
TC-29	Trigger: Evidence creates custody entry	Arrest above created	POST /api/evidence for that arrest	EvidenceCustody row "Collected and logged" created automatically	1 custody row created by trigger	Pass
TC-30	API: Add custody event	Evidence created	POST /api/evidence/:id/custody then GET	Two entries in chronological order	HTTP 201, 2 custody entries	Pass
TC-31	Trigger: Court proceeding moves case to In Court	Case 3 status Open	POST /api/proceedings for case 3	Case 3 status = In Court	case 3 = In Court	Pass
TC-32	Stored procedure: sp_close_case	Case 2 exists	POST /api/cases/2/close	Case 2 status = Closed and the change is logged	status=Closed, log row written	Pass
TC-33	Stored procedure: sp_criminal_history	Criminal 3 linked to case 3 in seed data	GET /api/criminals/3/history	Case list and arrests returned	HTTP 200, 1 case(s) in history	Pass
TC-34	Constraint: ON DELETE CASCADE on CriminalCase	Case created by the FIR test	Link criminal 3 to new case, delete the case	Junction rows removed automatically	junction rows 1 -> 0	Pass
TC-35	Constraint: ON DELETE SET NULL on Complaint.case_id	Complaint 1 was linked to the deleted case	Query Complaint 1 after case deletion	Complaint still exists with case_id NULL	complaint kept, case_id = NULL	Pass
TC-36	Reports: Dashboard totals	Seed + test data	GET /api/dashboard	Totals and chart data present	HTTP 200, totals.cases=3, upcoming=2	Pass
TC-37	Reports: Report: case-overview	Database populated	GET /api/reports/case-overview	HTTP 200 and a JSON array	HTTP 200, 3 row(s)	Pass
TC-38	Reports: Report: wanted-criminals	Database populated	GET /api/reports/wanted-criminals	HTTP 200 and a JSON array	HTTP 200, 0 row(s)	Pass
TC-39	Reports: Report: station-stats	Database populated	GET /api/reports/station-stats	HTTP 200 and a JSON array	HTTP 200, 2 row(s)	Pass
TC-40	Reports: Report: officer-workload	Database populated	GET /api/reports/officer-workload	HTTP 200 and a JSON array	HTTP 200, 3 row(s)	Pass
TC-41	Reports: Report: repeat-offenders	Database populated	GET /api/reports/repeat-offenders	HTTP 200 and a JSON array	HTTP 200, 0 row(s)	Pass
TC-42	Reports: Report: status-log	Database populated	GET /api/reports/status-log	HTTP 200 and a JSON array	HTTP 200, 5 row(s)	Pass
TC-43	Reports: Unknown report	None	GET /api/reports/nope	HTTP 404	HTTP 404	Pass
TC-44	Search: Global search	Seed data	GET /api/search?q=theft	Matching case returned	HTTP 200, 1 case match(es)	Pass

7.4 User-Interface Test Cases

Table 31 lists the 24 UI test cases. 24 of 24 passed. The screenshots in Chapter 6 were captured during this run.

Table 31: User-interface test cases and results (all executed)

ID	Feature	Preconditions	Input	Expected result	Actual result	Status
UI-01	Login page	Logged out	Open /	Hero, form and demo-account chips are shown	Form, hero text and demo chips visible	Pass
UI-02	Login error state	Logged out	admin / wrongpass, Sign in	Error banner "Invalid credentials", user stays on login	Banner "Invalid credentials" shown	Pass
UI-03	Login success	Seeded admin user	admin / admin123, Sign in	Dashboard with welcome banner is shown	Dashboard shown: "Welcome back, admin"	Pass
UI-04	Dashboard	Seed data	Open /	Stat cards, charts and upcoming hearings render	All three sections rendered	Pass
UI-05	Cases / FIR list page	Seed data	Open /cases	Page lists seeded rows (contains "FIR/2026/001")	Rows rendered from the database	Pass
UI-06	Complaints list page	Seed data	Open /complaints	Page lists seeded rows (contains "M. Patel")	Rows rendered from the database	Pass
UI-07	Criminals list page	Seed data	Open /criminals	Page lists seeded rows (contains "Vikram Rao")	Rows rendered from the database	Pass
UI-08	Arrests list page	Seed data	Open /arrests	Page lists seeded rows (contains "Theft (IPC 379)")	Rows rendered from the database	Pass
UI-09	Evidence list page	Seed data	Open /evidence	Page lists seeded rows (contains "Recovered mobile phone")	Rows rendered from the database	Pass
UI-10	Court Proceedings list page	Seed data	Open /proceedings	Page lists seeded rows (contains "Adjourned")	Rows rendered from the database	Pass
UI-11	Officers list page	Seed data	Open /officers	Page lists seeded rows (contains "R. Kumar")	Rows rendered from the database	Pass
UI-12	Stations list page	Seed data	Open /stations	Page lists seeded rows (contains "Central Station")	Rows rendered from the database	Pass
UI-13	Courts list page	Seed data	Open /courts	Page lists seeded rows (contains "District Court")	Rows rendered from the database	Pass
UI-14	Case detail page	Case 1 exists	Open /cases/1	Title, linked criminals, hearings, evidence and status history shown	All sections present	Pass
UI-15	Criminal profile page	Criminal 1 exists	Open /criminals/1	Profile, case history and arrests shown	Profile with both tables	Pass
UI-16	Add form with FK dropdowns	Admin	Arrests > Add new	Modal with Criminal, Case, Officer dropdowns populated	Modal open, dropdowns populated	Pass
UI-17	Chain-of-custody dialog	Evidence 2 has a custody row	Evidence > Custody	Dialog shows custody history	Custody history displayed	Pass
UI-18	Status filter	Case 1 is In Court	Cases > filter "In Court"	Only In Court cases remain	FIR/2026/001 shown, others hidden	Pass
UI-19	Empty state	None	Criminals > search "zzzz"	Friendly "No matching records" message	Empty state shown	Pass
UI-20	Reports page	Seed data	Open /reports	Report table plus Export CSV and Print buttons	Table and buttons present	Pass
UI-21	Global search	Seed data	Type "theft" in top search, Enter	Results page lists the matching case	Results page with FIR/2026/001	Pass
UI-22	Database error shown in UI	Officer 3 handles a case	Officers > Delete first officer	Red error banner explains the foreign-key violation; row remains	Banner shows foreign key constraint message	Pass
UI-23	Viewer role is read-only	Viewer login	Open Cases as viewer	No Add, Edit or Delete controls	No write controls rendered	Pass
UI-24	Mobile layout and drawer	Viewport 390x844	Open Cases, tap menu button	Drawer opens; page has no horizontal overflow	Drawer opened, no horizontal overflow	Pass

7.5 Summary of Results

Table 32: Summary of executed tests

Group	Cases	Passed	Failed
Authentication	6	6	0
Authorization	5	5	0
CRUD	6	6	0
Validation	6	6	0
Stored procedure	4	4	0
Trigger	5	5	0
API	1	1	0
Constraint	2	2	0
Reports	8	8	0
Search	1	1	0
User interface (system)	24	24	0
Total	68	68	0

7.6 Observations and Defects

- During development, several UI pages first failed to compile because of mismatched JSX tags after an edit; these were found by the build and by loading each page, and fixed before the final test run. The final run contains no failing case.
- **Behaviour that passed but is not ideal:** tests TC-18 and TC-19 pass because the invalid data was rejected and not stored, but the API answered with HTTP 500 instead of a 4xx validation status. This is recorded as a limitation (Section 8.4).
- The first version of the automated login-success UI test failed because the script typed into fields that were already filled; the test was corrected to use a fresh page. This was a test-script issue, not an application defect.

7.7 Tests Not Performed

The following were not executed and no results are claimed for them:

Table 33: Proposed tests that were not executed

Area	Proposed test	Reason not done
Unit tests	Isolated tests of React components and helper functions with a test framework.	No unit-test framework is configured in the project.
Load / performance	Response times and throughput with many users and large tables.	Out of scope for the demonstration data set; no figures are reported.

Area	Proposed test	Reason not done
Token expiry	Request with an expired JWT (8 hours).	Would need a clock change or a shorter test token.
Cross-browser	Firefox, Safari, Edge.	Only Chrome was used.
Accessibility audit	Screen-reader and contrast audit.	Not performed; only basic labelling was applied.
Security review	Penetration test, rate limiting, XSS review.	Out of scope for the course project.
Backup and restore	Restore the database from a dump.	Not part of the project.

Chapter 8: Results and Discussion

8.1 Summary of Implemented Features

Table 34: Objectives and the evidence that they were met

Objective (Section 1.4)	Result	Evidence
1. Centralized database with keys	13 tables, 16 foreign keys, unique and not-null constraints, 3 views.	Chapter 4; TC-21, TC-20
2. Digital management of criminals, cases, arrests, evidence	Full create, read, update and delete for all modules; chain of custody.	Chapter 6; TC-29
3. Court proceedings and instant reports	Hearings per case and court; six reports with CSV and print; dashboard.	TC-37 to TC-43; UI-20
4. Fast searching and cross-referencing	Per-module and global search; criminal and case detail pages.	TC-44, UI-21
5. Automation in the database	3 stored procedures and 4 triggers keep statuses consistent.	TC-26 to TC-32
6. Role-based access	Three roles enforced by API and reflected in the UI.	TC-08, TC-10, UI-23

8.2 Results Observed during Testing

All 44 API and database test cases and all 24 UI test cases passed in the final run (68 of 68). The most important behaviours confirmed were: the arrest trigger changes the criminal and the case in one statement; every status change is logged; a duplicate FIR number rolls back the whole FIR procedure; referenced parent rows cannot be deleted; cascade and set-null rules behave as designed; role restrictions are enforced by the API and reflected in the UI; and an SQL-injection string in the search box does not damage the database.

8.3 How the Application Addresses the Original Problem

Table 35: Problems from the project review and the corresponding solution

Problem	How PCRMS addresses it
Data duplication	One normalized database; UNIQUE constraints; related data referenced by key instead of copied.
Difficult searching	Search box on every list, a global search, a status filter and detail pages that gather related records.
Poor case tracking	Status field with fixed values, automatic transitions by triggers, a status-history table shown on the case page.
Time-consuming management	Forms with dropdowns for related records, one-click case closing, six ready-made reports with export.

8.4 Limitations and Unresolved Issues

The following limitations are known and were verified from the code or the tests:

- **Validation responses:** invalid or incomplete input returns HTTP 500 with the raw database message instead of a 400-level response. There is no server-side validation of formats (for example phone numbers, date ranges) beyond what the database enforces.
- **No user management:** there is no registration, password change or user administration screen; the three accounts come from the setup script.
- **Session storage:** the JWT is kept in localStorage and a development fallback secret is used if JWT_SECRET is not set. CORS is open to all origins. These settings are suitable for a demonstration, not for production.
- **No pagination:** list endpoints return all rows; text search uses LIKE with a leading wildcard, which scans the table. This is adequate for the demonstration data but will not scale to very large tables.
- **Audit scope:** the status log records what changed and when, but not which user changed it; other tables have no audit trail.
- **Photographs:** only a URL is stored; there is no upload or storage of image files.
- **Access scope:** every authenticated user can read every record; there is no per-station restriction.
- **Test coverage:** there are no unit tests and no load, cross-browser or accessibility tests (Section 7.7).
- **Platform coverage:** development and testing were done only on Windows 11 with Chrome.

Chapter 9: Conclusion and Future Scope

9.1 Summary of the Work

The team designed a thirteen-table relational schema in third normal form for police records, implemented it in MySQL with constraints, indexes, three views, three stored procedures and four triggers, built a REST API with role-based authentication in Node.js and Express, and built a responsive React interface covering all nine modules of the project plan. The system was exercised with 68 automated test cases, all of which passed, and every page was captured from the running application for this report.

9.2 Practical Value

PCRMS demonstrates how a DBMS can be the core of an application rather than a back-end afterthought. Integrity is enforced by the database; lifecycle rules run in procedures and triggers; the application adds convenience, security and presentation. For a police station the practical gains are a single searchable record of people, cases, arrests, evidence and hearings, an automatic and auditable case status, an accountable evidence trail and instant reports.

9.3 Limitations

The main limitations are listed in Section 8.4: error codes for invalid input, absent user-management screens, demonstration-grade session handling, no pagination and a limited audit trail. None of them affects the correctness of the data model.

9.4 Future Scope

The following are realistic next steps. None of them is implemented in the current version.

- Return 400-level validation errors with field-level messages, and add server-side format validation.
- Add user administration (create users, reset passwords, assign officers) and move the token to an httpOnly cookie; enforce a required JWT secret and restrict CORS.
- Add pagination, server-side sorting and full-text indexes for large data sets.
- Extend the audit trail to record the user for every change and to cover more tables.
- Allow photograph and document upload with secure file storage.
- Restrict data by station or jurisdiction.
- Add automated unit and end-to-end tests to a continuous-integration pipeline.

- Features from the project plan that remain future work: a mobile application, a cloud-hosted database, biometric integration, AI-based crime-pattern analysis and real-time notifications.

References

1. React Team, "React Reference Documentation," React. [Online]. Available: https://react.dev
2. Vite Team, "Vite Guide," Vite. [Online]. Available: https://vite.dev/guide/
3. Remix Software, "React Router Documentation," React Router. [Online]. Available: https://reactrouter.com
4. Motion, "Motion for React Documentation," Motion. [Online]. Available: https://motion.dev/docs/react
5. OpenJS Foundation, "Node.js Documentation," Node.js. [Online]. Available: https://nodejs.org/docs/latest/api/
6. OpenJS Foundation, "Express 4.x API Reference," Express. [Online]. Available: https://expressjs.com/en/4x/api.html
7. Oracle Corporation, "MySQL 8.4 Reference Manual," MySQL. [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/
8. mysql2 contributors, "MySQL2 Documentation," node-mysql2. [Online]. Available: https://sidorares.github.io/node-mysql2/docs
9. M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519
10. Auth0, "jsonwebtoken," GitHub repository. [Online]. Available: https://github.com/auth0/node-jsonwebtoken
11. D. Wirtz (dcodeIO), "bcrypt.js," GitHub repository. [Online]. Available: https://github.com/dcodeIO/bcrypt.js
12. Mermaid contributors, "Mermaid Documentation," Mermaid. [Online]. Available: https://mermaid.js.org
13. Google Chrome team, "Puppeteer Documentation," Puppeteer. [Online]. Available: https://pptr.dev
14. E. F. Codd, "A relational model of data for large shared data banks," Communications of the ACM, vol. 13, no. 6, pp. 377-387, 1970.
15. P. P. Chen, "The entity-relationship model: toward a unified view of data," ACM Transactions on Database Systems, vol. 1, no. 1, pp. 9-36, 1976.

Appendix A: API Endpoint Summary

All routes except POST /api/login require an Authorization: Bearer token. Viewers may use GET only; officers may use GET, POST and PUT; administrators may use all methods.

Table 36: Special API routes

Path	Method	Description
/api/login	POST	Authenticate and return token (public)
/api/dashboard	GET	Totals, cases by status and type, upcoming hearings
/api/reports/:name	GET	case-overview, wanted-criminals, station-stats, officer-workload, repeat-offenders, status-log
/api/fir	POST	Register FIR (sp_register_fir)
/api/cases/:id/close	POST	Close case (sp_close_case)
/api/cases/:id/detail	GET	Case with related records
/api/cases/:id/criminals	POST	Link a criminal to a case
/api/cases/:id/criminals/:cid	DELETE	Unlink a criminal
/api/criminals/:id/history	GET	Profile, case history, arrests
/api/evidence/:id/custody	GET, POST	Chain-of-custody entries
/api/search?q=	GET	Search criminals, cases, officers

Table 37: Generated CRUD routes (9 resources x 5 routes)

Path	Methods	Description
/api/stations	GET, POST	PoliceStation: list (supports ?q= search) and create
/api/stations/:id	GET, PUT, DELETE	PoliceStation: read one, update, delete (delete: admin only)
/api/officers	GET, POST	Officer: list (supports ?q= search) and create
/api/officers/:id	GET, PUT, DELETE	Officer: read one, update, delete (delete: admin only)
/api/criminals	GET, POST	Criminal: list (supports ?q= search) and create
/api/criminals/:id	GET, PUT, DELETE	Criminal: read one, update, delete (delete: admin only)
/api/cases	GET, POST	Cases: list (supports ?q= search and ?status=) and create
/api/cases/:id	GET, PUT, DELETE	Cases: read one, update, delete (delete: admin only)
/api/complaints	GET, POST	Complaint: list (supports ?q= search) and create
/api/complaints/:id	GET, PUT, DELETE	Complaint: read one, update, delete (delete: admin only)
/api/arrests	GET, POST	Arrest: list (supports ?q= search) and create
/api/arrests/:id	GET, PUT, DELETE	Arrest: read one, update, delete (delete: admin only)
/api/evidence	GET, POST	Evidence: list (supports ?q= search) and create
/api/evidence/:id	GET, PUT, DELETE	Evidence: read one, update, delete (delete: admin only)
/api/courts	GET, POST	Court: list (supports ?q= search) and create
/api/courts/:id	GET, PUT, DELETE	Court: read one, update, delete (delete: admin only)
/api/proceedings	GET, POST	CourtProceeding: list (supports ?q= search) and create

Path	Methods	Description
/api/proceedings/:id	GET, PUT, DELETE	CourtProceeding: read one, update, delete (delete: admin only)

Appendix B: Database Schema Reference

Table 38 lists every foreign key with its delete and update rule, as read from information_schema, and Table 39 lists the engine of each table.

Table 38: Foreign-key constraints

Table	Column	References	ON DELETE	ON UPDATE
AppUser	officer_id	Officer.officer_id	SET NULL	NO ACTION
Arrest	case_id	Cases.case_id	NO ACTION	NO ACTION
Arrest	officer_id	Officer.officer_id	NO ACTION	NO ACTION
Arrest	criminal_id	Criminal.criminal_id	NO ACTION	NO ACTION
Cases	officer_id	Officer.officer_id	NO ACTION	CASCADE
CaseStatusLog	case_id	Cases.case_id	CASCADE	NO ACTION
Complaint	station_id	PoliceStation.station_id	NO ACTION	NO ACTION
Complaint	case_id	Cases.case_id	SET NULL	NO ACTION
CourtProceeding	case_id	Cases.case_id	NO ACTION	NO ACTION
CourtProceeding	court_id	Court.court_id	NO ACTION	NO ACTION
CriminalCase	criminal_id	Criminal.criminal_id	CASCADE	NO ACTION
CriminalCase	case_id	Cases.case_id	CASCADE	NO ACTION
Evidence	arrest_id	Arrest.arrest_id	CASCADE	NO ACTION
EvidenceCustody	evidence_id	Evidence.evidence_id	CASCADE	NO ACTION
EvidenceCustody	officer_id	Officer.officer_id	NO ACTION	NO ACTION
Officer	station_id	PoliceStation.station_id	NO ACTION	CASCADE

Table 39: Storage engine and collation

Table	Engine	Collation
AppUser	InnoDB	utf8mb4_0900_ai_ci
Arrest	InnoDB	utf8mb4_0900_ai_ci
Cases	InnoDB	utf8mb4_0900_ai_ci
CaseStatusLog	InnoDB	utf8mb4_0900_ai_ci
Complaint	InnoDB	utf8mb4_0900_ai_ci
Court	InnoDB	utf8mb4_0900_ai_ci
CourtProceeding	InnoDB	utf8mb4_0900_ai_ci
Criminal	InnoDB	utf8mb4_0900_ai_ci
CriminalCase	InnoDB	utf8mb4_0900_ai_ci
Evidence	InnoDB	utf8mb4_0900_ai_ci
EvidenceCustody	InnoDB	utf8mb4_0900_ai_ci
Officer	InnoDB	utf8mb4_0900_ai_ci
PoliceStation	InnoDB	utf8mb4_0900_ai_ci

Appendix C: Additional Screenshots

The pages below complete the set of module screens (all captured in the same test run as Chapter 6).

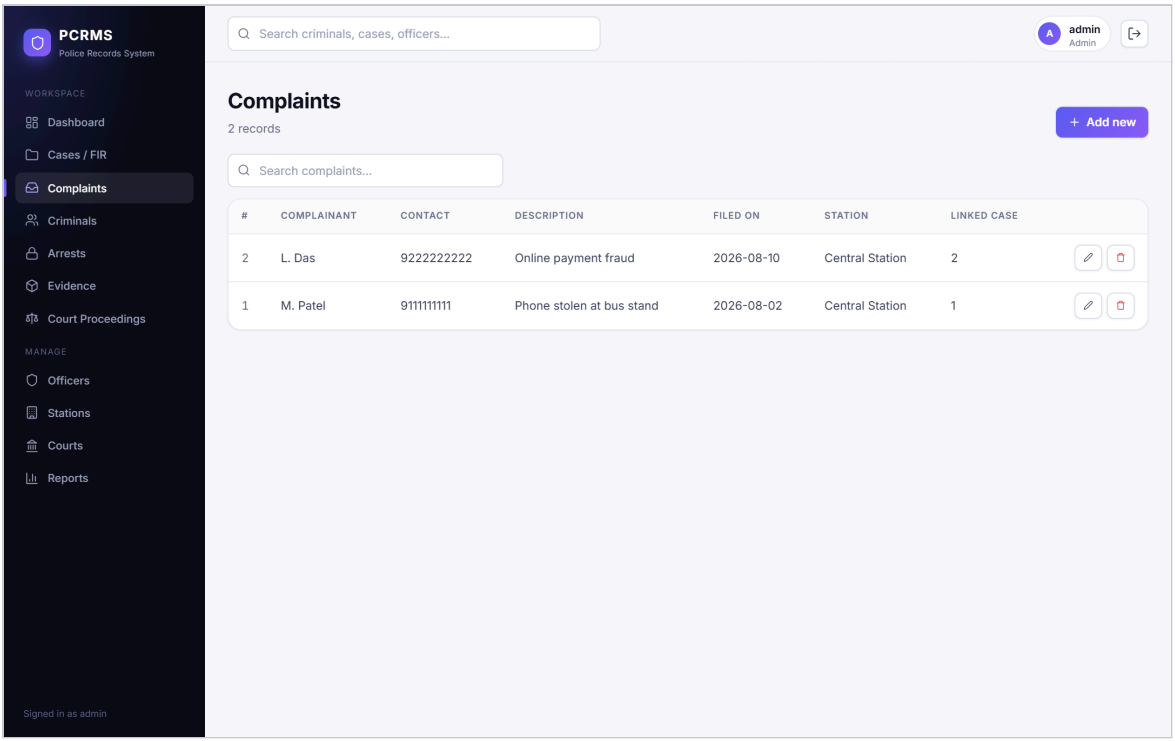


Figure 29: Complaints list

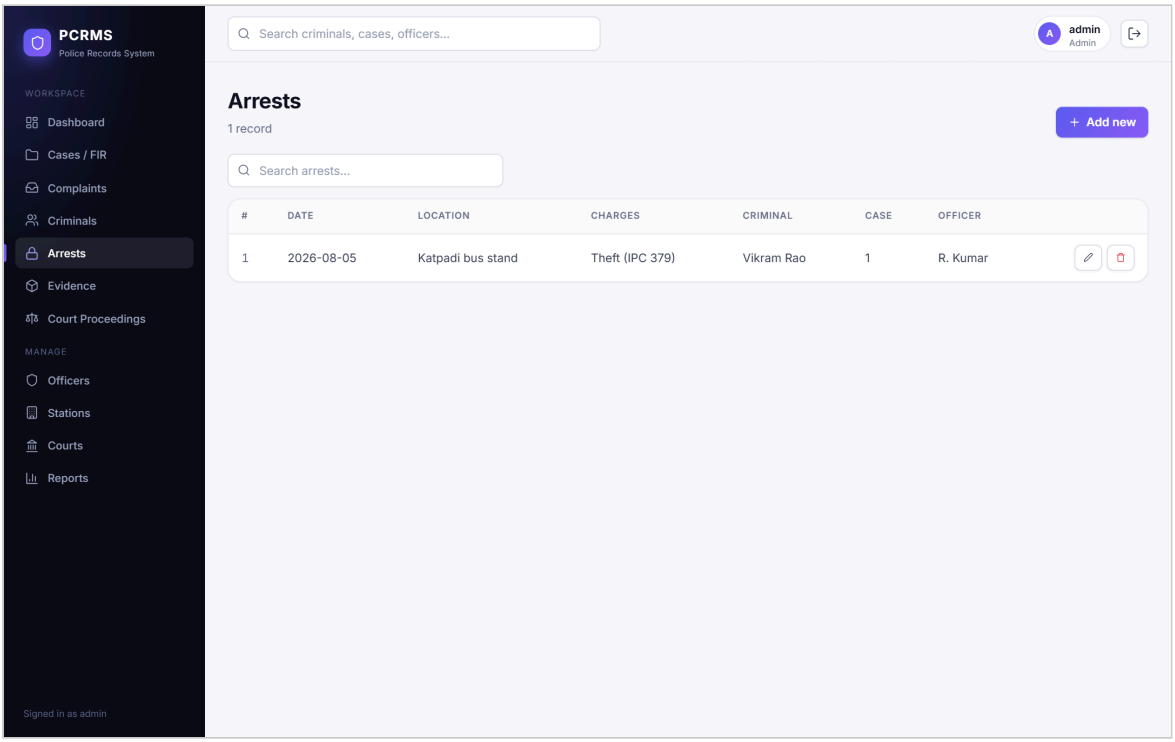


Figure 30: Arrests list

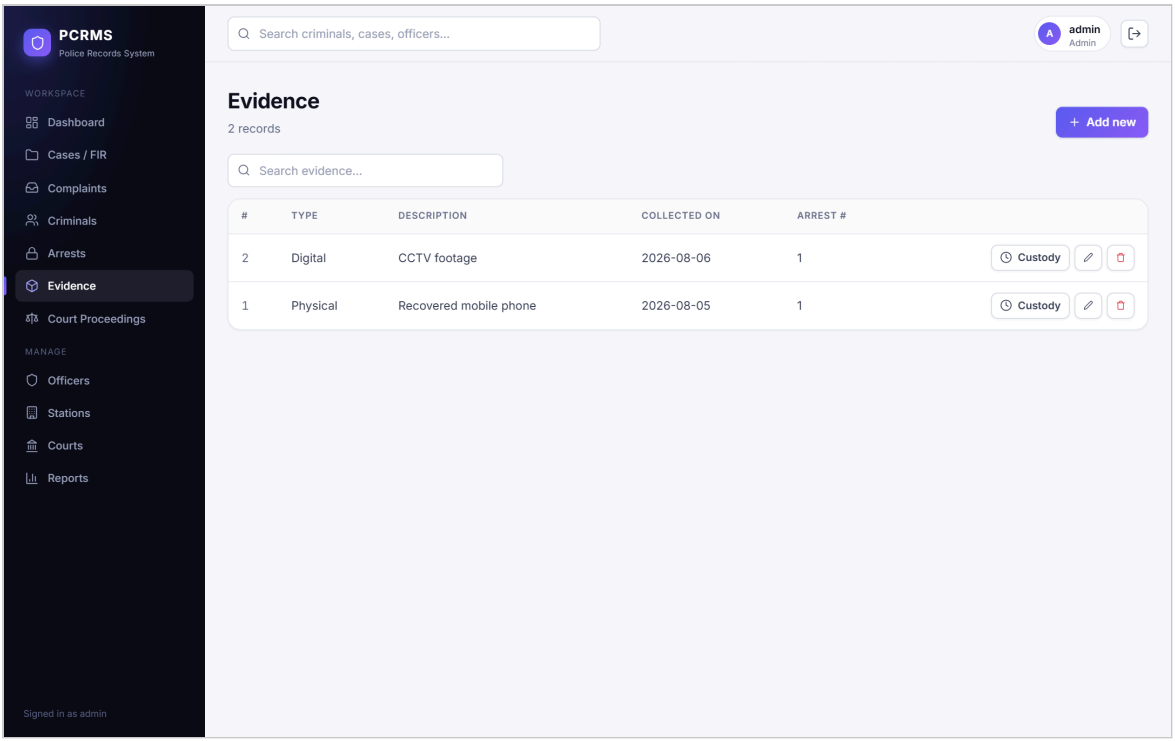


Figure 31: Evidence list

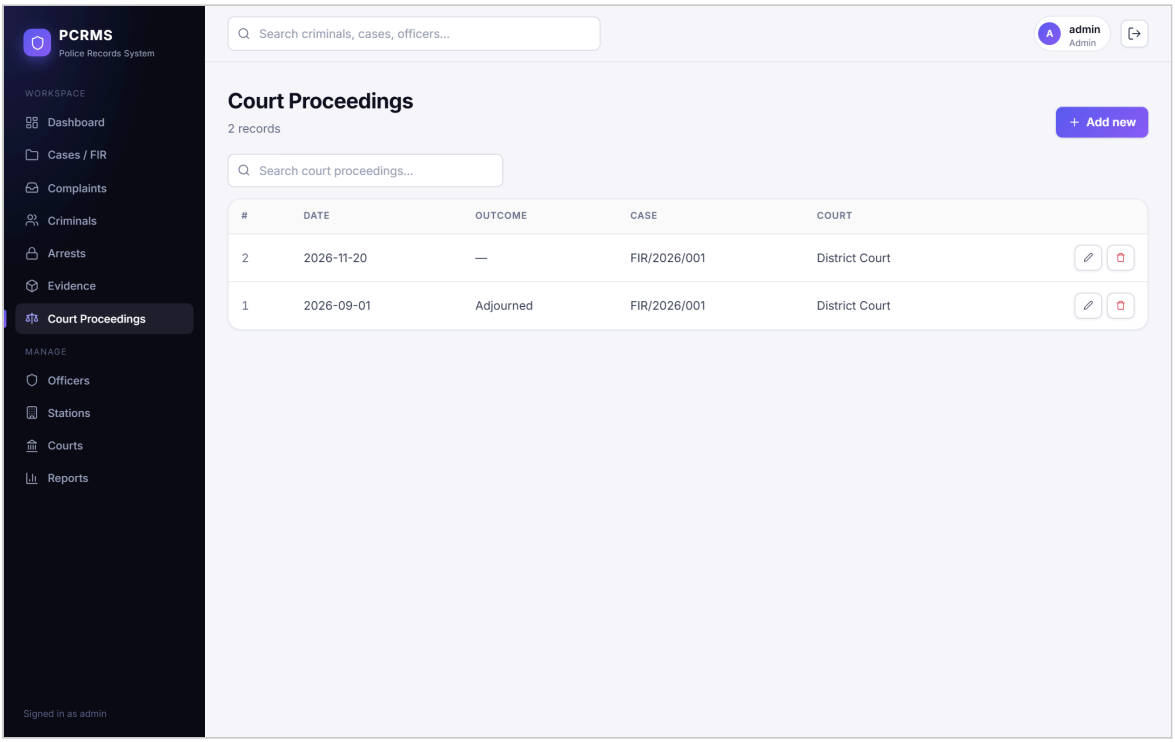


Figure 32: Court proceedings list

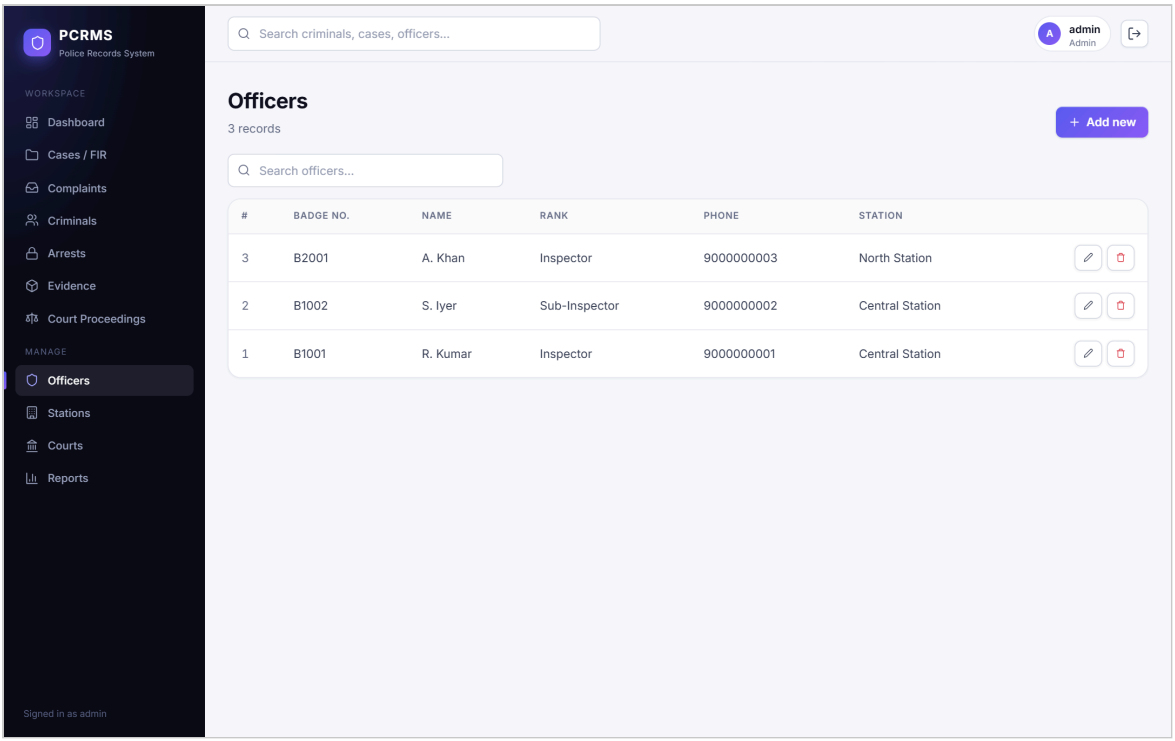


Figure 33: Officers list

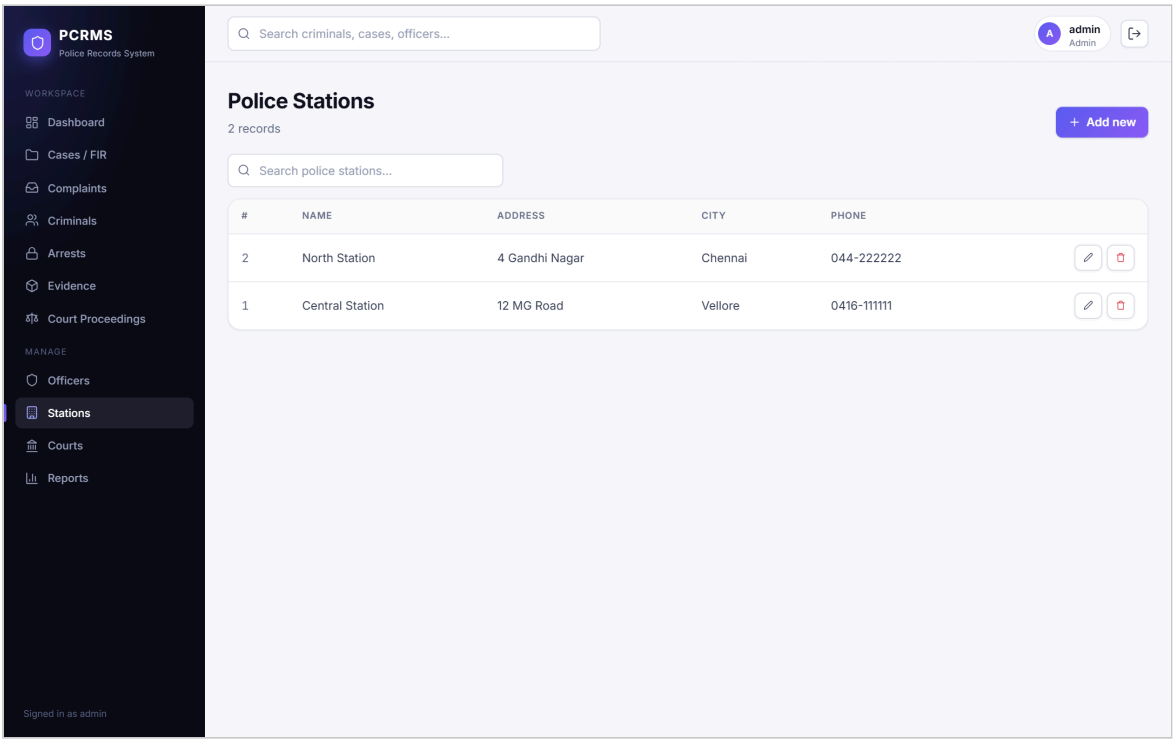


Figure 34: Police stations list

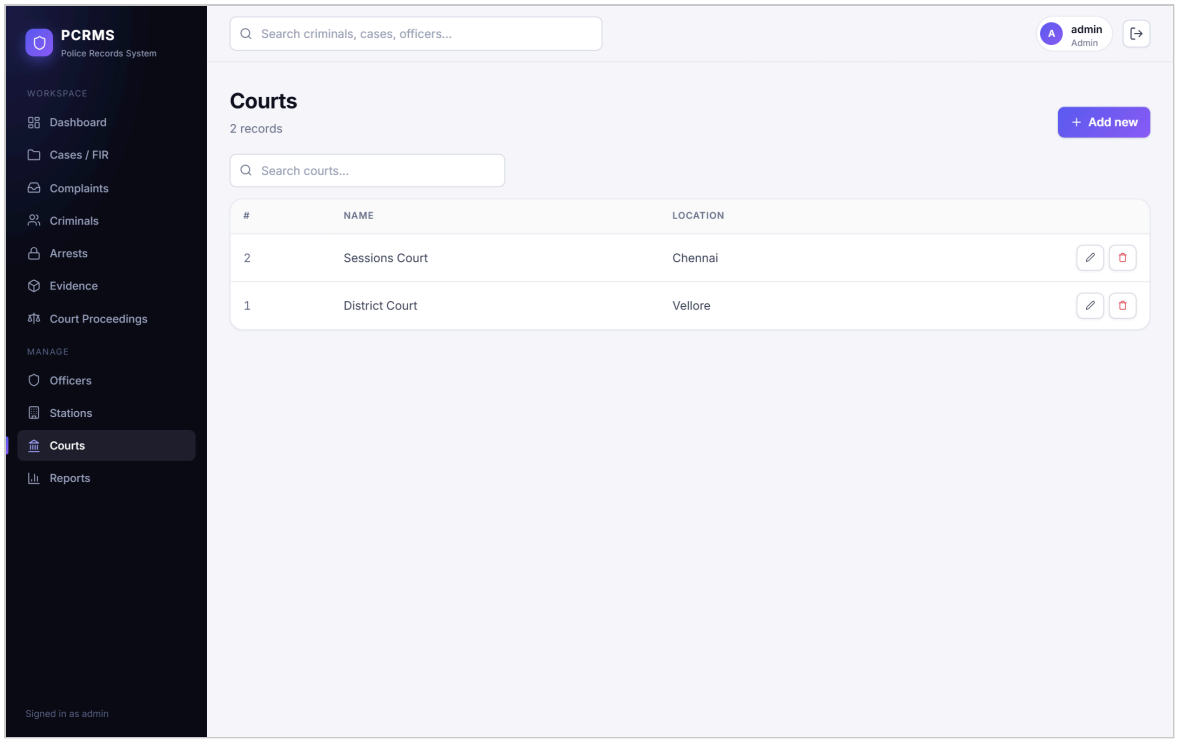


Figure 35: Courts list

Appendix D: Source Code and Configuration Notes

Selected excerpts are given in Chapters 4 and 5. Configuration notes:

- **Database connection (server/.env):** DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME, JWT_SECRET and PORT. The file .env.example lists the defaults. JWT_SECRET must be set to a strong value in any real deployment.
- **Vite proxy (client/vite.config.js):** requests starting with /api are forwarded to http://localhost:4000.
- **npm scripts:** server: setup-db, start, dev (node --watch); client: dev, build.
- **Automated tests:** node server/tests/run-tests.js writes its results to a JSON file given as the first argument.

Appendix E: Setup and Execution Instructions

1. Install Node.js 18 or later and MySQL 8, and start the MySQL server.
2. In the server folder copy .env.example to .env and set DB_PASSWORD (and JWT_SECRET).
3. In the server folder run: npm install, then npm run setup-db (creates the database, views, procedures, triggers, sample data and the three demo users), then npm start. The API listens on port 4000.
4. In the client folder run: npm install, then npm run dev. Open http://localhost:5173.
5. Log in with admin / admin123 (full access), officer / officer123 (no delete) or viewer / viewer123 (read only). These are demonstration credentials.
6. To run the automated API tests, run npm run setup-db and then node tests/run-tests.js in the server folder.

Appendix F: Glossary

Table 40: Glossary

Term	Meaning
FIR	First Information Report: the initial record of a reported offence from which a case is opened.
Chain of custody	Chronological record of who handled an evidence item and what was done.
Junction table	A table that resolves a many-to-many relationship into two one-to-many relationships.
Trigger	Database code that runs automatically after an INSERT, UPDATE or DELETE.
Stored procedure	Named, reusable SQL routine stored in the database.
View	Stored query that behaves like a virtual table.
Normalization	Organizing tables to reduce redundancy and update anomalies.
JWT	JSON Web Token: a signed token that carries identity and role claims.
RBAC	Role-based access control: permissions are assigned to roles, not individuals.
SPA	Single-page application: the browser loads one page and updates it with data from an API.